

Dan 2: Kodiranje i pisanje



Upotreba AI alata u studentskom istraživanju

AUTHOR

Luka Šikić, UNICATH

PUBLISHED

1. siječnja 2025.

Uvod

Jučer ste naučili koristiti umjetnu inteligenciju za pretraživanje literature, sintezu izvora i strukturiranje argumenata. Danas prelazimo na drugu polovicu istraživačkog procesa, onu koja od nejasne ideje pravi konkretan rezultat, kodiranje, analizu podataka i pisanje akademskog teksta. Ako je prvi dan bio o tome kako pronaći i organizirati znanje, drugi dan je o tome kako to znanje pretvoriti u empirijski dokazive tvrdnje i čitljiv tekst koji će kolege, mentori i čitatelji moći razumjeti, kritizirati i nadograđivati.

No prije nego što otvorite terminal ili editor, morate razumjeti nešto fundamentalnije. Računalni alati za kodiranje prošli su u posljednje četiri godine jednu od najdramatičnijih transformacija u povijesti softverskog inženjerstva. Prema aktualnim istraživanjima tržišta, više od 92 posto programera i podatkovnih znanstvenika integriralo je AI asistente u svoje svakodnevne radne tokove, čime je samo kodiranje prestalo biti isključivo manualna sintaktička aktivnost i prešlo u domenu orkestracije sustava i inženjeringa konteksta. Tržište AI alata za kodiranje, procijenjeno na preko 12,3 milijarde dolara do 2027. godine, iznjedrilo je nove metodologije rada, uključujući fenomen takozvanog vibe kodiranja, programiranja vođenog isključivo prirodnim jezikom, koje je proglašeno rječničkom riječi godine uslijed masovne adopcije. Rane analize pokazuju da čak 25 posto tehnoloških startupa novije generacije posjeduje repozitorije u kojima je više od 95 posto koda generirano isključivo putem umjetne inteligencije. Ovo nije budućnost. Ovo je sadašnjost u kojoj živite i u kojoj ćete raditi.

Ono što je 2021. godine bilo pametno dovršavanje redaka koda, tehnički napredak koji je malo promijenio dnevnu praksu programera, danas je autonomni sustav koji čita cijele projekte, piše skripte, pokreće ih, čita greške, ispravlja se i ustraje dok sve ne proradi. Eksplozija velikih jezičnih modela rezultirala je jasnom fragmentacijom tržišta na tri ključne paradigme, od kojih svaka zadovoljava specifične kognitivne i operativne potrebe, agencijsko programiranje, IDE integrirano programiranje i konverzijsko programiranje kroz LLM bilježnice. Ovaj dan prati tu evoluciju od najmoćnije paradigme prema najjednostavnijoj, počevši od agentskog kodiranja kroz terminalni interface, nastavljajući s IDE integriranom asistencijom, konverzijskim bilježnicama, i završavajući pisanjem akademskog teksta, jer čak i najbolja analiza ostaje beskorisna ako je ne znate artikulirati.

Svi primjeri koriste isti referentni istraživački projekt iz prvog dana, analizu vidljivosti Hrvatske narodne banke u medijskom diskursu o inflaciji, s korpusom od 31.536 članaka iz dnevnih i tjednih medija i makroekonomskim vremenskim serijama od siječnja 2015. godine do srpnja 2024. godine. Alati se mijenjaju, paradigme se mijenjaju, ali podaci i istraživačko pitanje ostaju isti, kako biste mogli susmisleno usporediti pristupe i razumjeti kada je koji alat najprimjerenije koristiti.

Prije nego počnemo

Provjerite da imate pristup alatima iz vodiča za instalaciju: pristup claude.ai s aktiviranom funkcionalnosti Claude Code, GitHub Copilot aktiviran kroz Student Developer Pack s vašom obrazovnom email adresom, i Positron IDE instaliran na vašem računalu. Ako nemate sve alate, radite u parovima s kolegom koji ima pristup.

1 Blok 2.1: Agentsko kodiranje s Claude Code 45 min

1.1 Paradigma agentskog kodiranja, kako su strojevi naučili programirati sami

U veljači 2025. godine tvrtka Anthropic objavila je alat pod nazivom Claude Code, sučelje naredbenog retka koja je omogućila nešto do tada nezamislivo za masovnog istraživača, da on u terminalu, običnim rečenicama na prirodnom jeziku, zada zadatak, a da umjetna inteligencija samostalno napiše kod, pokrene ga, pročitava poruku o pogrešci, ispravi se i ponovi cijeli ciklus dok analiza ne prođe besprijekorno bez ljudskog uplitanja. Do listopada iste godine funkcionalnost se proširila na web i mobilna sučelja, omogućujući pregledavanje koda i upravljanje projektima izvan striktnog terminalskog okruženja. Ovo nije bio inkrementalni napredak u softveru za programiranje, kao što je poboljšanje brzine ili dodatna opcija u dijalošnom okviru. Bio je to arhitektonski prekid s cijelom tradicijom kako ljudska bića, od Edsaca 1949. godine nadalje, komuniciraju s računalima, i upravo taj prekid definira kontekst u kojem vi danas radite s podacima i empirijskim istraživačkim pitanjima.

Da biste shvatili koliko je ovaj pomak radikalan, zamislite tvorničku proizvodnu liniju iz početka dvadesetog stoljeća. U tradicionalnom programiranju vi ste radnik koji stoji na svakoj postaji linije, na blagovanju s mjestom. Sami postavljate materijal, sami pritišćete gumb koji aktivira stroj, sami kontrolirate kvalitetu gotovog proizvoda, sami ga prenosite na sljedeću postaju. Ako stroj zaštekla, ako se materijal zapetlja, vi neposredno stajete tamo, tražite grešku, digli je analizirate, ispravljate je vlastitim rukama i nastavljate. Svaki korak zahtijeva vašu fizičku prisutnost, vašu budnost, vašu pažnju. U agentskom kodiranju vi ste voditelj tvornice koji ujutro daje jasne upute timu specijaliziranih robota, ne upute kako svaki robot mehanički funkcionira, nego upute što trebaju proizvesti. Velite im, «Napravite sto komada proizvoda X prema specijaciji Y, s tolerancijom od jedan milimetar na dimenziji, a ako materijal tipa A nije dostupan, koristite alternativu B, ali validirati kompatibilnost s preostalim komponentama». Roboti sami koordiniraju međusobni rad, sami dijele informacije o stanju tvornice, sami rješavaju zastoje, sami provjeravaju kvalitetu tijekom procesa, sami korigiraju brzinu linije ako je potrebno, a vi na kraju dana pregledate gotov proizvod, pročitate izvještaje o agencijama, i odlučujete je li sve zadovoljilo specifikaciju ili trebaju dodatne korekcije. Ne morate znati kako svaki stroj mehanički funkcionira na razini zupčanika i elektronike, ali morate znati što želite proizvesti, po kojim standardima, i što će se desiti ako ti standardi nisu zadovoljeni. Upravo tako funkcionira agentski sustav poput Claude Codea, struktura koju ćete koristiti danas i u budućnosti, kada god se trebate baviti analitičkim problemima veće kompleksnosti. Vi definirate analitički cilj, objekt od interesa, parametre analize i kriterije za uspjeh, a sustav autonomno piše kod, izvršava ga, detektira pogreške, čita poruke o greškama, interpretira što je pošlo po zlu, generira ispravke i reiterira dok model ili analiza ne zadovolji specifikaciju.

Ova paradigma predstavlja kulminaciju razvojnog puta koji je trajao četiri godine i prošao kroz tri jasno razlučive generacije, od kojih svaka donosi fundamentalni skok u načinu na koji strojevi percipiraju zadatak,

generiraju rješenja, i komuniciraju s ljudima koji ih upotrebljavaju. Razumijevanje te evolucije nije čista akademska vježba. To je razumijevanje koje vam omogućuje da znate kada je koji alat optimalan, što od njega trebate očekivati, i gdje su njegove granice.

1.2 Tri generacije AI kodiranja, od dovršavanja rečenica do autonomne analize

1.2.1 Prva generacija, 2021. godine, sustavi za automatsko dovršavanje sintakse

Prva generacija integracije umjetne inteligencije u programiranje uzela je maha tijekom 2021. godine i temeljila se na sustavima za automatsko dovršavanje, takozvani autocomplete, pokretanim ranim velikim jezičnim modelima kapaciteta do nekoliko milijardi parametara. Zamislite da sjedite u ispitnoj dvorani i pišete esej iz povijesti pod vremenskim pritiskom, te pored vas sjedi student koji je pročitao apsolutno sve udžbenike, sve povijesne Reference, sve memoare dostupne na engleskom jeziku, ali koji nikada nije bio na terenu, nikada nije razgovarao s ljudima koji su živi doživjeli događaje koje piše. Svaki put kad zastanete s perom, kada brzo razmislite što bi bila sljedeća rečenica, on vam šapne mogućnost kako bi sljedeća rečenica mogla završiti. Ponekad pogodi savršeno, ponekad predloži gramatički besprijeckornu glupost na razini faktičnog sadržaja, ali uvijek reagira samo na ono što ste upravo napisali, ne analizirajući značaj ili tijek cijele strukture vašeg eseja. Upravo tako su funkcionirali rani alati poput Kitea i inicijalnih verzija GitHub Copilota prije 2023. godine. Operirali su na reaktivnoj, bezdržavnoj arhitekturi, čekajući okidače temeljene na neposrednom unosu korisnika na tipkovnici, na detektiranju da je korisnik napisao komentar koji sugerira što dolazi zatim. Njihov primarni način rada sastojao se od predlaganja sljedećeg retka koda ili, u najboljem slučaju, sljedeće funkcije, analizirajući isključivo neposredni sintaktički kontekst unutar jedne datoteke otvorene u razvojnom editoru.

Statistički podaci iz tog razdoblja oslikavaju složenu, često razočaravajuću sliku. Do 2024. godine više od 62 posto programera koristi neki oblik AI alata za dovršavanje koda, što predstavlja veći udio od korisnika GitHub Copilota u samoj tvrtki GitHub. Unutar tehnoloških divova poput Googlea više od četvrtine cjelokupnog novog koda generirano je uz pomoć umjetne inteligencije, što sugerira da su alati tehnički dostali u sve proizvode. Ali stvarni učinci na produktivnost, mjereni kao čas potreban da se kod napiše, što je kod kvalitetan kada je napisan i koliko puta se vraća na reviziju, bili su dvosmisleni i često negativni. Prema izvještajima instituta DORA iz 2024. godine, koji analizira podatke od više od deset tisuća softverskih inženjera u Fortune 500 tvrtkama, usprkos masovnoj adopciji AI alata za dovršavanje, ukupna brzina isporuke softvera na globalnoj razini zapravo je zabilježila pad, a stabilnost sustava u produkciji se pogoršala. Zašto? Zato što su programeri provodili nerazmjerno mnogo vremena, dokumentacijski je pokazano u izvještajima, ispravljajući sintaktički točan ali logički i arhitektonski neispravan kod koji je generirao AI, kod koji se uopće ne snalazi u kontekstu većeg sistema. Sustav je mogao završiti rečenicu, sintaktički je bila potpuna, ali nije razumio njezino značenje, nije razumio kako se ta rečenica uklapa u ostatak programa. Rezultat je bio nešto poput studenta koji piše tečne, gramatički ispravne odgovore na ispitu, odgovore koji izgledaju impresivno na prvi pogled, ali koji odgovara na krivo pitanje, ili koji odgovara na ispravno pitanje ali s arhitektumom razmišljanja koja je neusklađena s dijelom znanja koji se testira.

Zašto bi vas kao studente i kao buduće istraživače ovo trebalo zanimati? Zato što ćete i danas, čak i u eri treće generacije koju ćemo odmah razmotriti, povremeno koristiti autocomplete sugestije u editoru, jer često su korisne, a napredni editore će vam ih nuditi. Trebate razumjeti njihova ograničenja na intuitivan način. Autocomplete vidi drveće, ne šumu. Kada vam predloži liniju koda za manipulaciju vremenskom serijom, ta linija može biti sintaktički savršena, kod će se kompajlirati bez greške, ali konceptualno pogrešna u kontekstu vaše analize, pogrešna u odnosu na hipotezu koju testirate ili na podatke koje analizirate. Ako analizirate vremensku seriju inflacije tijekom posljednje deset godina i autocomplete predloži funkciju za analizu presjeka, analizu različitih država u istoj godini, vi ste jedina linija obrane. Zato što će kod besmisleno proći bez sintaksne greške, ali će vam dati rezultate koji nemaju značenje.

1.2.2 Druga generacija, 2023. godine, kolaborativni modeli temeljeni na razgovoru

Tijekom 2023. godine industrija je zakoračila u eru kolaborativnog programiranja u paru, što se u engleskom žargonu naziva pair programmer era. Zamislite da umjesto studenta koji šapće završetke rečenica, sada sjedi pored vas pravi kolega, čovjek ili žena s iskustvom, s kojim možete razgovarati. Možete mu reći, «Napiši mi funkciju koja računa klizni prosjek za vremensku seriju s prozorom od dvanaest mjeseci, primjenjenu na mjesečne podatke o stopi inflacije, i ispiši rezultate takvim da mogu ih direktno ucrtati u graf». On će napisati cijeli blok koda, objasniti vam što svaka linija radi, objasniti izbore koje je učinio, odgovoriti na vaša potpitanja, sugerirati alternativne pristupe. Alati poput GitHub Copilot Chata, Codeiuma i Tabnine transformirali su korisničko iskustvo upravo na ovaj način, omogućujući konverzijsku komunikaciju s umjetnom inteligencijom unutar razvojnog okruženja koristeći prirodni jezik umjesto da je trebalo znati točnu sintaksu jezika za programiranje.

Skok je bio znatan i kvalitativno drugačiji od prve generacije. Umjesto da dovršava redak, model druge generacije mogao je generirati cijele složene blokove koda, često od pedeset do sto linija, na temelju tekstualnog opisa na nekoliko rečenica, demonstrirajući dublje razumijevanje namjere programera, analizirajući veće količine konteksta, do tada, u ratifikaciji datoteke, prilagođavajući se specifičnom kodu koji je već bio u projektu. Analitičari su ih počeli koristiti ne samo za pisanje nove logike, već i za objašnjavanje naslijeđenog koda koji nisu razumjeli, za generiranje testnih skripti, za pronalaženje specifičnih grešaka u dugoročnom kodu napisu prije deset godina.

Ipak, arhitektura druge generacije i dalje je zahtijevala stalni ljudski mikromenadžment, što znači da vi morate biti preokupljeni tehničkim detaljima procesa. Radni proces izgledao je ovako, i mnogi od vas će to prepoznati ako ste radili s ChatGPT-om. Vi opišete problem u chat sučelju, pričekate nekoliko sekundi da AI generira kod, ručno kopirate taj kod iz prozora za razgovor, ljepite ga u ciljanoj datoteku u editoru, pokrenete skriptu, vidite poruku o pogrešci, vraćate se u terminal, kopirate cijelu poruku o pogrešci, vraćate je u prozor za razgovor, zatražite ispravak, čekate, ponovno kopirate, ljepite. Ako je taj ciklus trebalo ponoviti pet puta za jednu analizu, a vaša analiza od eksplorativne do finalne registracije ima dvadeset koraka, računica brzo postaje neodrživa, i prepoznajete da ste proveli veći dio vremena s mehaničkim prenosima između prozora nego sa stvarnom razmišljanjem o analizi. Umjetna inteligencija bila je napredan asistent, ali čovjek je i dalje funkcionirao kao mehanički most između svake faze razvoja i izvršavanja koda. Zamislite da imate izvrsnog kuhara u restoranskoj kuhinji koji sprema jela prema vašim detaljnim uputama, ali vi morate osobno nositi svaki sastojak iz spremišta hladnjaka do kuhinje, svaku tavu od štednjaka do stola gdje se jelo služi, i svaki

prljava tanjur natrag u sudoper. Kuhar je izvrstan, njegova iskustva su valjana, ali logistika, fizički transport, je vaša, i upravo ta logistika troši većinu vašeg vremena i umora vas više od stvarnog rada.

1.2.3 Treća generacija, 2025. godine, doba autonomnih agentskih sustava

Tijekom 2025. godine industrija je doživjela ono što se najlakše može opisati kao prijelaz s ručnog mjenjača na autopilot, što je analogija poznata vozačima automobila cijelog svijeta. Treća generacija, era agentskog kodiranja, uvela je koncept agencije, sposobnosti umjetne inteligencije da neovisno razumije razvojne ciljeve na kontekstualnoj razini, autonomno zaobilazi prepreke s kojima se suočava tijekom razvoja, i ustraja u iterativnim pokušajima sve dok ne postigne potpuno funkcionalno rješenje koje ispunjava vaše specifikacije.

Razlika u odnosu na prethodne generacije je strukturalna na više dimenzija. Modeli poput Claude 3.7 Sonnet i Claude 4, s varijantama koje se razlikuju u razini konverzacijske agencije i strogosti rasuđivanja, posjeduju kontekstualne prozore koji dosežu preko 200.000 tokena, otprilike što je ekvivalent šezdeset stranica tiskane knjige. To je dovoljno velika memorija da omogućiti modelima da u svoju radnu memoriju učitaju arhitekturu cjelokupnih repozitorija, ne samo izoliranih datoteka. Ali ključna odlika nije samo veličina memorije, već sposobnost izravne akcije bez čekanja da čovjek završi klikanje i kopiranje. Umjesto da korisniku sugerira kod koji treba kopirati, agentski sustav posjeduje softverske ovlasti, privilegije, za čitanje repozitorija, modifikaciju datoteka, strukturiranje novih direktorija i, što je presudno, direktno izvođenje naredbi unutar terminala računala. Ovaj ciklus triju radnji - percipiranja podataka iz koda i grešaka koje sustav čita, kognicije ili razmišljanja što trebam napraviti, i akcije pisanja i pokretanja koda - omogućuje agentu da se u sebi, autonomno i bez prekida, korigira u petlji povratne informacije koja ne zahtijeva ljudsku intervenciju nakon što je upit postavljen.

Razmjer napretka ilustrira vanjski dostupan podatak o alatu Replit Agent, koji je javno praćen tijekom 2024. godine. U rujnu 2024. godine agent je mogao autonomno operirati oko dvije minute prije nego što bi trebao ljudsku pomoć, prije nego što bi trebao da će vas pitati za pojašnjenje ili da će zastati jer je došao do točke na kojoj više ne može inferencirati što trebate. Do rujna 2025. to se vrijeme produžilo na dvjesto minuta neprekinutog autonomnog rada. To je poboljšanje od sto puta u rasponu od jedne godine. To nije linearan napredak gdje se vremenska sposobnost agenta samo dvostruko ili trostruko povećala. Ovo je promjena agregiranog stanja sustava, fundamentalan skok u kvalitativnom ponašanju.

Tri generacije AI kodiranja u jednoj tablici

Dimenzija	Prva generacija (2021.)	Druga generacija (2023.)	Treća generacija (2025.)
Način rada	Autocomplete u editoru	Chat s generacijom blokova	Agentski s autonomnom egzekucijom
Jedinica analize	Jedan redak koda	Blok od 50-100 linija	Cijeli projekt i repozitorij
Kontekst	Trenutna datoteka	Chat prozor	200.000+ tokena perzistentne memorije
Izvršavanje koda	Nijedan (samo sugestije)	Korisnik ručno pokriva kod	Autonomno s terminalnom integracijom

Dimenzija	Prva generacija (2021.)	Druga generacija (2023.)	Treća generacija (2025.)
Povratna informacija	Nema	Korisnik mora kopirati greške	Agent čita greške i iterira
Ljudska intervencija	Česta (svaki redak)	Česta (između faza)	Rijetka (glavne odluke)
Adoption stopa	62% programera	Više od 50% IDE korisnika	Rastući, 2025+

Svaki korak u ovoj evoluciji reducira količinu mehaničkog posla koji pada na čovjeka, dok istovremeno povećava zahtjev za konceptualnim razumijevanjem onoga što čovjek od stroja traži i zašto zahtijeva upravo to.

1.3 Arhitektura Claude Codea, kako agent razmišlja i radi

Vratimo se na analogiju s tvornicom. Sada zamislite da vaš voditelj tima robota nije jedan robot, nego orkestar specijaliziranih strojeva koji se međusobno koordiniraju kroz centralni kontrolni sistem. Jedan robot je iskusni inženjer koji preuzima inicijativu i samostalno analizira korijene problema kada ga otkrije. Drugi je precizan tehničar koji brzo izvršava zadatke ali povremeno traži potvrdu od inženjera prije nego nastavi u drugom smjeru. Treći je koordinator koji prima dugoročne ciljeve, dekompozira složene probleme na manje, logički neovisne cjeline, raspodjeljuje ih specijaliziranim robotima, i nadgleda njihov napredak. Upravo tako funkcionira Claude Code, iako razina autonomije i karakteristike koordinacije ovise o temeljnom modelu koji pokreće sustav.

Model Claude 4 Opus funkcionira kao inženjer s inicijativom i iskustvom. U empirijskim testiranjima, kada bi naišao na grešku u bazi podataka, primjerice da program ne može pristupiti nekoj datoteci, Opus ne bi samo napisao provjeru za datoteku i nastavio kao da je problem riješen. On bi autonomno proveo analizu temeljnog uzroka, root cause analysis, sustavno razmišljajući što je učinilo da datoteka nije dostupna i je li to simptom većeg problema s dozvolama ili s putanjom. Implementirao bi rješenje kroz ispravan API sučelja operacijskog sustava, te proaktivno dodao indikatore napretka kako bi znali što se zbiva, konfiguracijsko sučelje kako bi mogli prilagoditi ponašanje, i naprednu validaciju unosa kako bi se slični problemi ne bi mogli ponovo dogoditi. To je refleksija ponašanja iskusnog profesionalca koji ne čeka da mu verbalno objasnite što je očito. Model Claude 4 Sonnet, s druge strane, dizajniran je kao oprezni suradnik, sličan mlađem inženjeru sa solidnim temeljima ali s manjim iskustvom. Djeluje brzo, isporučuje funkcionalan kod, ali frekventnije zahtijeva potvrde, mikrokorekcije i pojašnjenja namjere, funkcionirajući slično kompetentnom mlađem kolegi koji želi biti siguran da ide u pravom smjeru prije nego investira vrijeme u detaljnu implementaciju.

Najnaprednija značajka alata je sposobnost višeagentske orkestracije, što se na engleskom zove multi-agent orchestration. Kada postavite izrazito složen analitički problem, problem koji nije samo primjena već zahtijeva razmišljanje na više razina, možete pokrenuti glavnog agenta koji automatski dekompozira zadatak u hijerarhiju podzadataka. Vanjski gledano, vi potencijate jedan upit, ali interno, glavni agent tada instancira, stvara, višestruke paralelne sesije, takozvane podagente, u odvojenim granama projekta. Dok jedan podagent refaktorira sigurnosne module i provjerava da su svi stringovi validhirani, drugi simultano razvija testne skripte s Comprehensive test coverage, treći razvija dokumentaciju API sučelja. Glavni agent kontinuirano

nadgleda njihov rad, rješava konflikte ako dva podagenta trebaju istu datoteku, i spaja rješenja u jedinstvenu cjelinu. Za ekonometrijski projekt koji trebate napraviti, ovo znači da jedan podagent može čistiti podatke, transformirati varijable, obaviti sve pripremne korake, dok drugi istovremeno piše i testira vizualizacijske funkcije, a treći specificira regresijske modele s odgovarajućim kontrolama, sve koordinirano bez vašeg uplitanja u mehaniku procesa.

Temeljni problem konverzacijskih AI modela je oduvijek bio gubitak konteksta između sesija, nešto poput razgovora s kolegom koji svaki dan dolazi na posao bez sjećanja na jučerašnji rad ili na dogovore iz prethodnog tjedna. Vi trebate ponovno objasniti kontekst, ponovno reći koji su vam ciljevi, ponovno specificirati kako trebate da podaci izgledaju. Claude Code rješava ovaj izazov uvođenjem perzistentne memorije, persistent memory, funkcionalnosti koja je strukturna i centralna. Sustav kreira i kontinuirano ažurira datoteke unutar korijenskog direktorija projekta, specifično datoteke imena `CLAUDE.md` i `MEMORY.md`, u kojima se ne pohranjuje samo generiranih kod, što bi bilo Trivialno, već kognitivni kontekst iz razgovora. Te datoteke sadrže projektne smjernice i konvencije, stilove formatiranja koda koje trebate slijediti, ranije donesene arhitektonske odluke i obrazloženja zašto su donesene, pa čak i specifične terminalske naredbe potrebne za uspješno prevođenje i pokretanje projekta, isprintane informacije o verzijama biblioteke koja trebate. Kada nakon tjedana pauze započnete novu sesiju jer ste bili fokusirani na pisanje rada ili ste odmorili od kodiranja, agent automatski asimilira cjelokupni povijesni kontekst iz tih datoteka bez potrebe da mu ponavljate sve. Nema zamornog ponovnog objašnjavanja pravila. Nema potrebe da ponovno opisujete podatke i varijable.

Nadalje, arhitektura je dizajnirana oko otvorenog standarda zvanog Protokol konteksta modela, Model Context Protocol ili MCP. Zamislite da vaš agent osim pristupa vlastitoj radionici, vlastitoj datotečnoj strukturi i vlastitim alatima, ima i ključeve od arhiva, od knjižnice, od konferencijske dvorane. Kroz MCP, Claude Code može autonomno pretraživati dokumentaciju pohranjenu na Google Driveu, čitati kontekstualne rasprave na Slacku ili Teams kanalima gdje su vaši kolege diskutirali problem, ažurirati status zadataka unutar Jire ili Linear alata, pristupiti vanjskim bazama podataka kroz API sučelja. Transformacijski potencijal ovakve integracije potvrđuju interni podaci tvrtke Anthropic za 2025. godinu, prema kojima je pravni odjel tvrtke, koristeći više od 800 implementiranih AI agenata s MCP integracijom, skratio vrijeme pregleda ugovora i usklađenosti s regulatornim okvirom s prosječnih dva do tri dana na manje od 24 sata, što je očekivano, jer agenti mogu direktno pristupiti svim relevantnim dokumentima i regulativnim okvirima.

Zašto bi vas kao istraživače ovo trebalo zanimati? Zato što perzistentni kontekst i integracija s vanjskim alatima znače da vaš analitički projekt više nije izoliran u jednom skriptu ili jednom prozoru za razgovor. Agent razumije cjelokupnu povijest vašeg rada, pristupa izvorima podataka koji žive izvan vašeg računala, primjerice baze podataka na institutu, i održava konzistentnost kroz mjesece razvoja. Kada na sastanku mentor ili komisija pita zašto ste odabrali baš tu specifikaciju modela umjesto alternative, agent ima zapisano obrazloženje u memoriji projekta s datumom kada je odluka donesena i razlozima. To nije opcija ili nice-to-have značajka. To je standard profesionalnog rada u 2025. godini.

1.4 Demonstracijski tijek rada, makroekonomska analiza u pet koraka

Evolucija agentskog kodiranja nije limitirana na kreiranje softverskih aplikacija ili web stranica. Njezina je primjena revolucionirala polje znanosti o podacima, statističke analize i ekonometrijske obrade podataka, gdje

su tradicionalno bili potrebni mjeseci rada i duboko tehničko znanje. Kako bi se ilustrirala puna tehnička i kognitivna snaga alata Claude Code, dekonstruiramo demonstracijski empirijski proces od pet koraka. Referentno istraživanje fokusira se na mjerenje utjecaja medijske vidljivosti Hrvatske narodne banke na makroekonomska inflacijska očekivanja, na analizu kako često se HNB spominje u medijima, u kojem se kontekstu spominje, i kako to korelira s time što ljudi očekuju da će biti stopa inflacije u budućnosti. Ono što ovaj proces čini paradigmatiskim za eru agentske inteligencije jest činjenica da se svih pet složenih faza analize, od učitavanja sirovih podataka u različitim formatima do konstrukcije multiverzuma specifikacijskih krivulja i analiza robusnosti, izvršava isključivo kroz konverzaciju na prirodnom jeziku, bez da istraživač napiše ijednu liniju koda vlastitom rukom.

1.4.1 Demo: Pet koraka analitičkog pipelinea s Claude Code

Korak 1. Učitavanje podataka i autonomna eksplorativna analiza.

Temelj svake robustne ekonometrijske obrade je osiguravanje integriteta sirovih podataka, što često znači da trebate objasniti da su svi podaci ispravni i konzistentni. U tradicionalnom okruženju, analitičar bi morao ručno pisati funkcije za čitanje i parsiranje CSV datoteka, rezolvirati probleme s graničnicima između polja, zaglavlje u datotekama, osigurati točno mapiranje Unicode kodiranja znakova, snadbiti se rutinama za identifikaciju nepostojećih ili pogrešno formatiranih opservacija, što je obično zauzima nekoliko sati kada rade prvi put. U agentskom tijeku rada, korisnik pokrene proces jednostavnom naredbom na prirodnom jeziku, tražeći od Claude Codea čitljivu analizu podataka unutar specifičnog direktorija. Agent autonomno, bez ljudske intervencije, piše, izvršava i validira skripte koje učitavaju tablične podatke, provode dubinsku eksplorativnu analizu, identificiraju ključne varijable i njihove svojstva poput vremenskih serija indeksa potrošačkih cijena, medijskog indeksa HNB-a, tečaja valute, te automatski provjeravaju konzistentnost tipova podataka kroz sve varijable. Ukoliko naiđe na problem tijekom tog procesa, primjerice datume unesene kao tekstualne stringove umjesto kao datumske varijable, agent bez čekanja ljudske naredbe detektira pogrešku, diagnosticira uzrok, piše ispravak i ponovno pokreće analizu kako bi potvrdio da je problem riješen.

Korak 2. Čišćenje podataka i priprema panel strukture.

Sirovi makroekonomski podaci, posebno kada dolaze iz različitih izvora kao medijski članci i baze podataka, obiluju asimetrijama, različitim frekvencijama objava, duljinama vremenskih serija, i ekstremnim vrijednostima znane kao outliers. Analitičar putem prirodnojezikovnog prompta zadaje parametre i zahtijevajući filtriranje specifičnog vremenskog prozora, primjerice 2015. do 2024., i pripremu podataka za ekonometrijsku obradu. Agentski sustav ovdje pokazuje analitičku zrelost i razumijevanje domenske problematike jer ne primjenjuje destruktivne operacije naslijepo bez konteksta. Ako korisnik postavi upit o postupanju s ekstremnim opservacijama, primjerice mjesecu kada je bilo izvanredan gospodarskog šoka, agent generira opcije umjesto automatskog brisanja, nudeći argumentaciju o tome kako logaritamska transformacija varijabli, primjerice, stabilizira varijancu i omogućuje interpretaciju koeficijenata u terminima elastičnosti koja je često od interesa. Kada podaci dolaze u različitim frekvencijama, dnevna baza medijskih tekstova naspram mjesečnih objava cijena, agent autonomno programira funkcije za agregaciju, interpolaciju ili resampling, i spaja sve varijable u jedinstven, pročišćen analitički okvir koji je strukturiran kao balansirani panel spreman za modeliranje.

Korak 3. Napredna vizualizacija s dualnom y-osi.

Prije procjene regresijskih modela, znanstvena metodologija i dobra praksa nalažu vizualnu inspekciju trendova, što pomaže otkriti strukturalne lomove, nestacionarnost ili sezonalnost. Zadatak je tehnički zahtjevan i često zahtijeva znanje o grafičkim bibliotekama: trebate prikazati kretanje dviju posve različitih vremenskih serija, inflacije i medijskog indeksa HNB-a, na istom grafu. Inflacija se izražava u niskim postotnim poenima, obično između nule i šest posto, dok medijski indeks operira na znatno višoj apsolutnoj ljestvici, ponekad između pet tisuća i dvadeset tisuća. Standardni graf s jednom y-osi komprimirao bi jednu varijablu do gotovo nevidljivosti. Agent demonstrira duboko poznavanje sintakse `ggplot2` ekosustava u R-u, biblioteke koja je standard za statistiku. Budući da `ggplot2` po dizajnu otežava kreiranje dvostrukih osi jer je to često loša praksa u podatkovnoj vizualizaciji, agent implementira algebarsku transformaciju, pronalazeći optimalan faktor skaliranja, dijeleći vrijednosti medijskog indeksa s izračunatim koeficijentom unutar `geom_line()` sloja, te aktivirajući argument `sec.axis` unutar `scale_y_continuous()` s inverznom formulom koja vraća vrijednosti na originalnu ljestvicu. Istodobno, prepoznajući da vizualne korelacije na grafu mogu biti zavaravajuće, agent automatski izračunava i ispisuje rezultate Pearsonove korelacijske analize kao validacijski mehanizam koji potvrđuje da korelacija postoji ili ne postoji.

Korak 4. Rigorozna statistička inferencija: OLS i Newey-West HAC procjenitelji.

Temeljna metoda ekonometrijskog modeliranja je procjena metodom običnih najmanjih kvadrata, Ordinary Least Squares ili OLS, opisana jednadžbom

$$y_t = \beta x_t + u_t$$

gdje y_t označava stopu inflacije u mjesecu t , x_t medijski indeks HNB-a, a u_t rezidualnu pogrešku ili disturbance. OLS model izgrađen na vremenskim serijama podlozan je kršenju temeljnih pretpostavki Gauss-Markovljevog teorema, specifično pretpostavki o odsustvu autokorelacije i homoskedastičnosti. Inflacija je po prirodi autoregresivni proces, što znači njezina razina u razdoblju t snažno korelira s razinom u $t - 1$ i $t - 2$ jer inflaciju čine procesi s inercijom. Oslanjanje na klasičnu formulu za varijancu

$$\text{var}(\hat{\beta}) = \sigma^2 (X'X)^{-1}$$

bi proizvelo podcijenjene standardne greške, zbog toga što formula ignora autokorelaciju, što bi pak proizvedlo umjetno visoke t-vrijednosti i pogrešno zaključivanje o statističkoj značajnosti koeficijenta. Agent autonomno detektira potrebu za HAC, Heteroskedasticity and Autocorrelation Consistent procjeniteljem, u obliku Newey-West standardnih grešaka, procjeditelja koji je robustan na probleme s autokorelacijom i heteroskedastičnošću. Agent integrira R pakete `sandwich` i `lmtest`, kalibrira parametar truncacije, truncation parameter, prema heurističkim pravilima kao što je Stock-Watson pravilo za izbor broja lags, te prezentira valjane, ispravljene p-vrijednosti koje vi možete interpretirati kao malu ili veliku vjerojatnost da je koeficijent nula kad on zapravo nije nula.

Korak 5. Analiza multiverzuma i specifikacijska krivulja.

Umjesto da rezultate prikaže na jednoj preferiranoj specifikaciji modela, ono što bi bila klasična naivna praksa, agent autonomno generira analizu multiverzuma modela, analizu svih mogućih specifikacija, koristeći tehniku analize specifikacijske krivulje, Specification Curve Analysis, prema Simonsohnu,

Simmonsu i Nelsonu iz 2020. godine. Kroz R paket `stability` uz biblioteke `tidyverse`, `lfe` i `broom`, agent iterira kroz sve logične kombinacije bazičnih kontrola, permutabilnih varijabli i struktura fiksnih efekata. Ako su vam bazične kontrole demografske varijable i makroekonomski pokazatelji, permutabilne varijable su alternative koje mogu biti uključene ili ne, onda agent testira sve kombinacije. Svaki model je usklađen s Newey-West standardnim greškama. Konačni ispis je grafički prikaz od dvaju panela. Gornji panel prikazuje točkaste procjene primarnog regresora, koeficijenta na medijskom indeksu, uz intervale pouzdanosti, poredane po veličini učinka od najmanjeg prema najvećem, što vam omogućuje da vidite je li efekt robustan ili ovisi o specifikaciji. Donji panel matrično ilustrira komponente svake specifikacije, koje su kontrole uključene, što vam omogućuje da vidite zakonitosti. Cijeli proces, koji bi klasično zauzeo tjedne akademskog rada od više osoba, uključujući briljantnog analitičara koji poznaje sve te metode, svodi agentski sustav na zadatak od nekoliko minuta.

💡 Što se zapravo dogodilo

U pet koraka demonstracije nijedna linija koda nije napisana ručno. Istraživač je komunicirao na prirodnom jeziku, koristeći hrvatsku ili englesku, opisujući što želi postići, ne kako postići. Agent je autonomno odabirao pakete, decisions o bibliotekama koje trebate, pisao je funkcije, izvršavao ih, interpretirao greške, pisao ispravke i iterirao do ispravnog rezultata. Ovo ne znači da razumijevanje koda postaje nepotrebno. Znači da se fokus pomiče s mehanike pisanja sintakse na strategiju analize, na razumijevanje što trebate znati o vašim podacima.

1.5 Kako opisati analizu kada ne znate programirati, inženjerstvo promptova za istraživače

Transformacija paradigme uslijed uvođenja alata poput Claude Codea sa sobom donosi fundamentalnu promjenu na razini potrebnih ljudskih vještina. Interakcija s algoritmima više ne zahtijeva pedantno poznavanje računalne sintakse, ni memoriranje naziva funkcija, ni znanje o sintaksnim greškama koje su tek od važnosti zato što se sustav javi. Međutim, ova apstrakcija mehanike koda u prvi plan stavlja nužnost ovladavanja preciznom semantikom, domenskim vokabularom i strukturiranom logičkom komunikacijom, procesom koji se u engleskom žargonu industrije naziva prompt engineering ili prompt crafting. U profesionalnom žargonu, sposobnost delegiranja cijelih analitičkih cjelina umjetnoj inteligenciji putem prirodnog jezika označena je kolokvijalnim terminom vibe coding, što je besmisleno rečeno, ali proces upravljanja modelima treće generacije zahtijeva zapravo formalnu metodološku rigoroznost i primjenu definiranih komunikacijskih strategija.

Zamislite da ste upravitelj velikog istraživačkog tima s deset članova, ali 80 posto vaših članova ne govore isti jezik kao vi, oni govore Mandarin ili Arapski ili neki treći jezici. Komunikacija ide preko prevoditelja koji je izvrstan, ali nije pametan. Ako prevoditelju kažete vagu reč, «napravi nešto s onim podacima što smo učitali», rezultat će biti nepredvidiv. Možda će prevoditelj prosljediti samo «napravi nešto s podacima» i članovi tima će biti zbunjeni. Ako kažete «koristeći mjesečne podatke o inflaciji od siječnja 2015. do srpnja 2024. za Hrvatsku, procijeni linearni regresijski model u kojem je zavisna varijabla stopa inflacije, mjerena kao godišnja promjena indeksa potrošačkih cijena, a nezavisna varijabla je medijski indeks naslova o HNB-u, s robusnim Newey-West standardnim greškama korigiranim za autokorelaciju prvog reda i heteroskedastičnosti, i

prikaži rezultate s intervalom pouzdanosti od 95 posto», prevoditelj može precizno proslijediti vašu namjeru članovima tima i mogu početi s radom. Ista logika vrijedi za komunikaciju s agentskim sustavima.

Tri aksioma definiraju učinkovitu komunikaciju s agentskim sustavima. Prvo, opisujete krajnji cilj, krajnjeg rezultat koji trebate, umjesto da mikroupravlјate metodama na razini koda. Ključna pogreška neiskusnih korisnika je težnja kontroli na sintaktičkoj razini, na razini kako se točno kod napiše. Upute poput «kreiraj varijablu x čuvanjem vrijednosti, primijeni petlju koja iterira kroz redake, i zatim učitaj funkciju lm()» suzbijaju kognitivni kapacitet modela jer mu ne govore što zapravo trebate. Ispravan prompt artikulira željeni domenski ishod na konceptualnoj razini, «Procijeni utjecaj medijske izloženosti na inflaciju, osiguravajući da model sadrži robusne standardne greške korigirane za autokorelaciju, kako bi rezultati bili čvrsti». Agent se tada oslanja na svoju bazu znanja, svoju obuku, i samostalno odabira najprimjerenije pakete i pristupe. Zašto? Zato što agent ima bolje znanje nego što ga viimate o tome što je najbolja praksa za taj zadatak.

Drugo, iscrpno definirajte podatkovni kontekst i postavite granice na ono što agentski sustav smije raditi. Jedan od primarnih uzroka halucinacija kod jezičnih modela, od pogrešnih faktičnih tvrdnji do besmislenih brojeva, jest nedostatak eksplicitnog konteksta. Trebate navesti nazive ključnih varijabli s točnim imenom kao što se pojavljuje u vašoj datoteci, primjerice `CPI_index` ili `HNB_media_index`, trebate opisati njihovo značenje u domaćem jeziku, primjerice da je indeks potrošačkih cijena postotni omjer promjene, a ne apsolutna vrijednost u kunama, trebate objasniti očekivane odnose među varijablama i znanja bez kojega bi analiza bila besmislena. Granice kontroliraju domet djelovanja agenta i sprječavaju pogrešne odluke, primjerice «Prilikom čišćenja podataka, ne smiješ izbacivati ekstremne vrijednosti koje padaju u razdoblje pandemije, već ih tretiraj kao validne informacije jer su reprezentativne za neobično razdoblje». Ova granica sprječava da agent sazna grešno od vas što trebate.

Treće, implementirajte iterativne provjere i lanac rasuđivanja, chain-of-thought, što znači da trebate zahtijevati od agenta da objasni što radi. Rad s autonomnim agentima nosi rizik pretvaranja znanstvenog procesa u crnu kutiju gdje znate što ide u input i što izlazi kao output, ali kako se između tih točaka desilo, neznate. Prompt uvijek mora sadržavati imperativ za dekonstrukciju koraka, «Za svaku transformaciju podataka i svaki odabir parametra, pruži obrazloženje zašto je ta odluka optimalna u odnosu na alternativne pristupe». Ova zahtjev čini proces transparentnim i omogućuje vam da provjerite logiku agenta.

Učinkovitost komuniciranja kao analitička vještina

Duži, repetitivni upiti, upiti koji su zbunjenije i ne trebali ne samo što povećavaju troškove korištenja API-ja, što će vas finansijski boljeti, već zauzimaju dragocjen prostor unutar kontekstualnog prozora modela, što znači da agent ima manje mjesta da sâm razmisli. Izostavljanjem nepotrebnih fraza i sažetim strukturiranjem granica poboljšavate kvalitetu, stabilnost i brzinu odziva. Korištenje tehnika poput dohvaćanja uz generiranje, Retrieval-Augmented Generation ili RAG, omogućuje da se specifična baza znanja vaše organizacije, primjerice prethodni analitički kodovi ili metode, inkorporira u tijek razmišljanja umjetne inteligencije, smanjujući potrebu za opširnim ručnim uvođenjem domenskih informacija u svaki pojedinačni upit.

1.5.0.1 Praktična vježba (5 min)

Studenti u parovima dekompoziraju vlastiti istraživački projekt iz prvog dana u pet preciznih promptova za agentski sustav. Svaki prompt opisuje jedan korak analitičkog procesa bez referenciranja specifične tehnike ili biblioteke. Primjer za prvi prompt: «Učitaj mjesečne podatke o inflaciji i medijskom indeksu,

provjeri njihove tipove podataka, identifikuj bilo koje nedostajuće vrijednosti, i prikaži deskriptivnu statistiku za sve varijable». Ne trebate reći «koristi `read_csv()`, provjeri `typeof()`, koristi `is.na()`». Agent zna. Vaš posao je znati što trebate vidjeti. Drugi prompt: «Pročisti podatke uklanjanjem bilo kojih opservacija s nedostajućim vrijednostima, provedi logaritamsku transformaciju na sve ekonomske varijable, i kreiraj novu vremensku varijablu». Treći: «Kreiraj graf koji prikazuje trendove obje varijable kroz vrijeme, koristeći odgovarajući faktor skaliranja ako se ljestvice znatno razlikuju». Četvrti: «Procijeni linearni regresijski model s robusnim standardnim greškama». Peti: «Testiraj robusnost rezultata kroz analizu specifikacijske krivulje, varirajući specifikaciju modela i prikazujući sve procjene». Fokusira se na logiku analize, ne na kod.

2 Blok 2.2: IDE integracija s GitHub Copilot i Positron 45 min

2.1 IDE paradigma, fina kontrola i taktička iteracija

Za razliku od terminalno orijentiranih i izrazito autonomnih agenata koji rade iza zatvorenih vrata, integrirano razvojno okruženje (IDE) s ugrađenom AI podrškom zadržava apsolutni fokus na ljudskom programeru kao glavnom pokretaču svake akcije. Zamislite da umjesto voditelja tvornice koji ujutro daje upute robotima i navečer čita izvještaje, vi sjedite u pilotskoj kabini s golemom instrumentnom pločom ispred sebe. Svaki instrument pokazuje što se zbiva u realnom vremenu. Svaki prekidač zahtijeva vašu ruku. Autopilot vam šapće sugestije, ali vi upravljate avionom. Upravo tako funkcionira IDE integrirana AI asistencija. Vi pišete kod red po red, a umjetna inteligencija vam pomaže da svaki red bude bolji, brži i točniji. Ne prepuštate kontrolu. Ne gubite vidljivost. Ali zato trošite više kognitivne energije jer morate čitati, evaluirati i odobravati svaki prijedlog koji sustav generira.

Ova paradigma optimizirana je za taktičko, mikrorazinsko izvršavanje, za generiranje pojedinačnih funkcija izravno u editoru, za promptno ispravljanje sintaktičkih pogrešaka, za prevođenje algoritama iz jednog programskog jezika u drugi, za pisanje testova i za iterativno poboljšanje čitljivosti koda. Ovu sferu predvode ekstenzije poput GitHub Copilota, ugrađene u popularne uređivače, te specijalizirani AI nativni uređivači poput Cursora i Windsurfa koji su uspostavili nove standarde ergonomije kodiranja. Ono što te alate čini posebnima jest njihova dvojna, hibridna arhitektura koja neprekidno radi u pozadini. Ta arhitektura kombinira izrazito brze, minijaturene modele s niskom latencijom zadužene za automatsko dovršavanje sljedeće linije u stvarnom vremenu s većim, sofisticiranijim modelima koji se aktiviraju tek kada korisnik zatraži dublju analizu putem ugrađenog razgovornog panela. Ne morate znati koji se model aktivira u kojem trenutku. Ali morate znati da postoje dva sloja asistencije, brzi koji reagira dok tipkate i duboki koji reagira kada ga pitate.

2.2 Od monolitnog RStudija prema poliglotskom Positronu

Dugogodišnja dominacija RStudija u domeni podatkovne znanosti temeljila se na njegovoj uskoj, gotovo simbiotskoj integraciji s programskim jezikom R. Međutim, suvremeni analitički zahtjevi, koji sve više isprepliću R i Python unutar istog istraživačkog projekta, otkrili su strukturna ograničenja monolitnih

arhitektura. RStudio je dizajniran tako da se cjelokupno korisničko sučelje i svi pozadinski procesi izvode unutar jednog jedinog R procesa. U slučaju fatalne pogreške u kodu, beskonačne petlje ili preopterećenja memorije prilikom učitavanja iznimno velikih makroekonomskih skupova podataka, ruši se cjelokupno razvojno okruženje zajedno sa svim nesačuvanim rezultatima. Zamislite da imate radionicu u kojoj su svi strojevi spojeni na jednu utičnicu. Kada jedan stroj preoptereći mrežu, nestaje struja u cijeloj radionici i sve se zaustavlja, svi strojevi, sve svjetiljke, sve ventilacije. Upravo tako funkcionira monolitna arhitektura. Gubitak radnog okruženja u trenucima visoke kognitivne koncentracije predstavlja ozbiljnu prepreku u produktivnosti profesionalnih analitičara.

Positron predstavlja radikalni arhitektonski iskorak. Razvijen od strane tvrtke Posit (nekadašnji RStudio), Positron je razvojno okruženje izgrađeno povrhu infrastrukture Visual Studio Code (Code OSS), platforme koju koristi oko milijun softverskih inženjera diljem svijeta. Temeljna prednost ove arhitekture leži u odvajanju korisničkog sučelja od izvršnog koda. Positron operira s pozadinskim jezgrama (kernels) za svaki pojedini jezik, što znači da komunikacija s interpreterima za R i Python odvija se neovisno. Vraćajući se na radionicu, zamislite da sada svaki stroj ima vlastiti osigurač. Kada jedan stroj preoptereći svoju liniju, iskače samo njegov osigurač. Ostali strojevi nastavljaju raditi neometano. Analitičar jednostavno restartira kernel koji se srušio dok mu sučelje ostaje netaknuto sa svim otvorenim datotekama i rezultatima. Python je u Positronu tretiran kao jezik prvog reda, potpuno ravnopravan R-u, čime se stvara istinsko poliglotsko okruženje prilagođeno modernim radnim tokovima u podatkovnoj znanosti.

Vizualno i ergonomske prijelaz s RStudijom na Positron gotovo je nevidljiv. Zamislite da ste godinama vozili pouzdan automobil koji savršeno poznajete, svaki prekidač na mjestu, sve funkcionalnosti dostupne, sve intuitivno. Ali taj automobil nema modernu elektroniku, nema navigacijski sustav s inteligentnim rutama, nema senzore za parkiranje, nema adaptivni tempomat. Positron je novi model istog proizvođača koji vizualno izgleda poznato, ima isti raspored pedala i isti osjećaj volana jer je inženjerski tim čuvao intuicije koje poznajete iz RStudija, ali donosi cijelu generaciju elektronike koja vas čini bržim i sigurnijim analitičarem. Konkretno, Positron zadržava prepoznatljivi raspored panela iz RStudija. Konzola se i dalje nalazi u donjem dijelu ekrana gdje vidite rezultate, preglednik okruženja (Environment) omogućava stalan uvid u sve varijable u memoriji, grafički pregledač prikazuje vizualizacije. Ali ti paneli sada rade na VS Code infrastrukturi koja otvara vrata cijelom tržištu ekstenzija, uključujući GitHub Copilot, GitLens za praćenje promjena koda, Rainbow CSV za čitljivost tabularnih podataka i stotine drugih korisnih alata.

Značajna konceptualna razlika očituje se i u upravljanju projektima. Dok se RStudio primarno oslanjao na `.Rproj` datoteke za definiranje radnog okvira, Positron uvodi koncept radnih prostora (workspaces) preuzet iz VS Code paradigme. Radni prostor je analogan bilo kojoj mapi na lokalnom računalu, čime se eliminira potreba za kreiranjem specifičnih projektnih datoteka. Dodatno, Positron svjesno napušta praksu automatskog spremanja `.RData` datoteka u radni prostor prilikom izlaska iz aplikacije. Ova promjena rješava jedan od najčešćih uzroka grešaka u istraživačkom radu, kontaminaciju globalnog okruženja zaostalim varijablama iz prethodnih sesija, čime se forsira praksa pisanja reproduktibilnog koda koji se može izvesti od nule. Navigacija i upravljanje postavkama odvijaju se kroz paletu naredbi (Command Palette), centralizirani tekstualni alat na vrhu ekrana koji zamjenjuje višestruke padajuće izbornike modernijim pristupom.

Usporedba arhitektura RStudio i Positron

Komponenta	RStudio	Positron
Arhitektura	Monolitni R proces	VS Code (Code OSS) kernel arhitektura
Stabilnost	Pad R procesa ruši cijeli IDE	Pad kernela ne utječe na sučelje
Projekti	<code>.Rproj</code> datoteke, automatski <code>.RData</code>	Radni prostori bazirani na mapama, čisto okruženje
Ekstenzije	Ograničen ekosustav	Potpun pristup VS Code tržištu
Jezici	R primarni (Python preko <code>reticulate</code>)	R i Python kao ravnopravni jezici prvog reda
AI integracija	Ograničena nativna podrška	Duboka integracija (Copilot, Positron Assistant)

Zašto Positron umjesto RStudija za ovaj tečaj? RStudio je izvrstan alat s godinama razvoja i može biti ispravan izbor za učenje, ali nema izvornu, nativnu podršku za GitHub Copilot i druge AI ekstenzije. Positron zadržava poznati raspored podatkovne znanosti s RStudio sučeljem, ali otvara vrata cijelom VS Code ekosustavu. Za studente koji prelaze s RStudija, prijelaz je gotovo nevidljiv jer su paneli na istim mjestima, ali dobiti je značajna jer imate AI asistenciju integriranu izravno u editor.

2.3 GitHub Copilot, tri načina rada i akademski pristup

Studenti i nastavno osoblje viših učilišta, uključujući studente Sveučilišta u Zagrebu (@unizg.hr) i Hrvatskog katoličkog sveučilišta (@unicath.hr), ostvaruju pravo na besplatan pristup GitHub Copilot Pro inačici kroz program GitHub Student Developer Pack. Proces aktivacije započinje registracijom na edukacijskom portalu GitHub Education, nastavlja se verifikacijom akademskog statusa dodavanjem službene sveučilišne email adrese i učitavanjem dokaza o statusu poput potvrde fakulteta, te završava konfiguracijom sigurnosnih postavki. Posebno je važno omogućiti blokiranje prijedloga koda koji u potpunosti odgovaraju javno dostupnim repozitorijima, čime se drastično umanjuje rizik od nenamjernog plagiranja, te isključiti opciju koja GitHubu dopušta korištenje vašeg koda za treniranje modela, čime se štiti intelektualno vlasništvo istraživanja.

GitHub Copilot unutar IDE-a radi u tri jasno različita načina rada, od kojih svaki pokriva drugačiju fazu vašeg razmišljanja dok pišete kod. Prvo, linijsko predlaganje (inline suggestions) koje predlaže kod dok tipkate, gledajući što ste napisali do sada i predlažući sljedeću liniju ili blok. Ovo je onaj brzi šegrt koji vam dodaje alat u milisekundama, koji ponekad pogodi savršeno a ponekad treba korekciju. Pritisnete Tab za prihvaćanje ili Esc za odbijanje. Ovaj mehanizam je idealan za kompletiranje sintakse, imenovanje varijabli i pisanje repetitivnog koda. Drugo, razgovorni panel (Chat panel) gdje možete postavljati pitanja o kodu u prirodnom jeziku i dobiti odgovore od većeg, sofisticiranijeg modela koji ima više vremena za razmišljanje i može analizirati širi kontekst. Primjerice, možete pitati «Zašto ovaj model javlja grešku o singularnoj matrici?» ili «Kako da optimiziram ovu for petlju?». Treće, iterativno uređivanje (Edits) gdje model može napraviti usmjerene izmjene

u više datoteka odjednom, prikazujući razliku između starog i novog stanja kroz razlikovni prikaz (diff view) prije nego što prihvatite promjenu. Ovaj mehanizam drastično smanjuje brzinu u usporedbi s potpuno autonomnim agentima, ali maksimizira sigurnost. To je cijena transparentnosti. Ali ta cijena je dobra investicija.

2.4 Positron Assistant, domenski specifična nadgradnja

Dok GitHub Copilot pruža izvrsne općenite mogućnosti kodiranja, Positron uvodi vlastitu domenski specifičnu nadgradnju u vidu Positron Assistanta. Zamislite razliku između generičkog liječnika opće prakse i specijalista koji poznaje vašu kompletnu medicinsku dokumentaciju. Generički AI klijent unutar IDE-a primarno ima pristup statičkom kodu spremljenom na disku, čita vaše skripte ali ne zna što se događa u memoriji dok radite. Međutim, većina rada u podatkovnoj znanosti događa se interaktivno, u efemernome stanju gdje se podaci dinamički učitavaju, transformiraju i filtriraju u radnoj memoriji dok se modeli vizualiziraju i prilagođavaju u hodu. Positron Assistant dizajniran je upravo za takve radne tokove. On koristi napredne mehanizme pozivanja alata (tool-calling) kako bi izravno pristupio Positronovim specifičnim značajkama.

Kada korisnik postavi upit, Positron Assistant ne čita samo kod. On ima kontekst o učitanim varijablama u pregledniku okruženja, može vidjeti metapodatke, imena stupaca i tipove podataka unutar otvorenih podatkovnih okvira te može očitati povijest rezultata iz R konzole. Zbog toga, kada analitičar zatraži generiranje koda za specifičnu analizu inflacije, asistent eksplicitno zna nazive varijabli u memoriji, što drastično smanjuje stopu halucinacija i predlaganja nepostojećih funkcija. Dodatno, Positron Assistant omogućuje fleksibilnost u odabiru jezičnih modela u pozadini, dopuštajući korisnicima da konfiguriraju modele poput Claudea 3.5 Sonnet, OpenAI modele ili sam GitHub Copilot, ovisno o specifičnim potrebama zadatka i organizacijskim sigurnosnim politikama.

2.5 Kognitivni rizici, tehnički dug i cijena autonomije

Potpuno prepuštanje kontrole autonomnim agentima nosi empirijski dokazane rizike koje ne smijete zanemariti. Longitudinalne studije koje su analizirale primjenu AI agenata u razvoju softvera kroz repozitorije otvorenog koda ukazuju na pojavu značajnih izazova vezanih uz kvalitetu. Iako agenti donose snažno početno ubrzanje u generiranju koda, oni istovremeno mogu inducirati dugoročni tehnički dug. Analize provedene na AIDev skupovima podataka pokazuju da se u repozitorijima s visokim udjelom agentski generiranog koda upozorenja statičke analize povećavaju za približno 18 posto, a kognitivna kompleksnost koda raste za čak 39 posto. Ovi podaci jasno demonstriraju da delegiranje cjelokupne logike autonomnim sustavima, bez strogog iterativnog nadzora, rezultira kodom koji je teži za održavanje, teži za debugiranje i teži za nadogradnju, čak i kada su početne prednosti u brzini očigledne. Usko grlo produktivnosti seli se s mehaničkog pisanja koda na koordinaciju i nadzor nad agentom.

Za studente i istraživače, posebice u ekonometrijskom modeliranju gdje je točnost specifikacije modela kritična i gdje svaka transformacija podataka mora biti teorijski opravdana, IDE pristup je neusporedivo povoljniji. Umjesto da agentu prepustite potpunu autonomiju nad odabirom varijabli, odabirom procjenitelja i metodama evaluacije, vi koristite alate unutar Positrona kako biste premostili jaz između teorijskog znanja i programske implementacije. Ovaj simbiotski proces znatno smanjuje kognitivno opterećenje potrebno za prisjećanje specifične sintakse, dok vi i dalje suvereno vladate metodološkim okvirom. AI preuzima izvođenje

redundantnih blokova sintakse, ali vi u svakom trenutku ostajete kognitivno sidro koje verificira semantiku i teorijsku konzistentnost statističke inferencije.

2.6 Demo s HNB makroekonomskim podacima

2.6.1 Pet koraka iterativnog kodiranja s HNB podacima

HNB redovito prati i objavljuje opsežne baze podataka o inflaciji, koristeći Indeks potrošačkih cijena (CPI) kao opću nacionalnu mjeru i Harmonizirani indeks potrošačkih cijena (HICP) za komparativnu analizu na razini europodručja. Rad s ovim podacima zahtijeva poznavanje specifičnog konteksta, primjerice uvođenja eura u siječnju 2023. u uvjetima globalno visoke inflacije te značajnih izmjena u izračunu nacionalnog CPI-ja od siječnja 2026. s implementacijom nove ECOICOP 2 klasifikacije.

Korak 1. Učitavanje podataka i osnovna vizualizacija. Tipkate komentar koji opisuje cilj, primjerice `# Load Croatian inflation and HNB media data from 2015 to 2024`. Copilot čita komentar i predlaže `read_csv()` poziv iz `readr` paketa umjesto bazne `read.csv()` funkcije jer je brži, eksplicitno parsira tipove podataka i robusnije tretira nestale vrijednosti. Model također sugerira upotrebu paketa [here](#) za definiranje putanja čime se osigurava prenosivost koda. Pritisnete Tab, podatci su učitani. Zatim napišete komentar za vizualizaciju i linijsko predlaganje nudi kompletnu `ggplot2` specifikaciju histograma s prilagođenom temom.

Korak 2. Deskriptivna statistika s `gt` paketom. Sirovi statistički izlazi iz konzole nisu prihvatljivi u profesionalnom izvještavanju. U razgovornom panelu opisujete «I want a formatted table with summary statistics for all variables, with column headers in Croatian and right-aligned numbers». Copilot generira potpunu sintaksu koristeći `gtsummary` paket koji automatski prepoznaje distribuciju varijabli i izračunava odgovarajuće mjere središnje tendencije i disperzije, s funkcijom `modify_header()` koja prevodi nazive stupaca na hrvatski. Svaki broj u tablici deterministički je povezan sa sirovim podacima, čime se ostvaruje temeljni postulat reproduktibilnosti.

Korak 3. Vizualizacija vremenske serije. Počinjete pisati `ggplot(data, aes(x = date, i` i linijsko predlaganje se aktivira. AI asistent, prepoznajući iz konteksta da se radi o datumskoj osi i varijabli inflacije, automatski sugerira cjelokupnu specifikaciju grafa uključujući `geom_line()` za vremenski trend, formatirane datumske skale koje inače zadaju glavobolje analitičarima i smislene oznake osi. Često je predložak dovoljno dobar da trebate samo malo prilagodbe, potvrđujući Pareto princip gdje AI obavlja 80 posto posla u 20 posto vremena.

Korak 4. OLS regresija s Newey-West robusnim greškama. Obična metoda najmanjih kvadrata (OLS) daje nepristrane procjene regresijskih parametara čak i kada su prisutne heteroskedastičnost i serijska korelacija, ali standardne greške postaju pristrane što čini testove značajnosti nevaljanima. U razgovornom panelu opisujete «Estimate an OLS model with Newey-West standard errors to account for autocorrelation in a time series, and show the summary». Copilot generira cjevovod koji koristi paket `sandwich` za procjenu Newey-West matrice kovarijance i paket `lmtest` za provođenje Waldovih testova s ispravljenom matricom. Funkcija `NeweyWest` automatski primjenjuje napredne postupke poput

prethodnog izbjeljivanja (prewhitening) i odabira optimalnog odmaka. Vi se fokusirate na teorijsku specifikaciju i interpretaciju, ne na sintaksu.

Korak 5. Iterativno poboljšanje kroz panel analizu. Kada se analitički okvir proširi s Hrvatske na komparativnu analizu skupine zemalja srednje i istočne Europe, javlja se problem neopažene heterogenosti entiteta. Koristite linijska predlaganja za postavljanje specifikacije s fiksnim efektima unutar paketa `plm`, a zatim se obraćate Positron Assistantu s pitanjem kako testirati izbor između fiksnih i slučajnih efekata. Asistent generira Hausmanov test, objašnjava interpretaciju i, ako p-vrijednost sugerira odbacivanje slučajnih efekata, preporučuje dodatne korekcije poput Driscoll-Kraay robusnih grešaka za panel podatke s vremenskom i prostornom korelacijom.

2.7 Komparacija doživljaja između paradigmi

Razlika između agentskog kodiranja iz prethodnog bloka i IDE integriranog kodiranja nije u kvaliteti koda koji nastaje, jer oba sustava mogu generirati funkcionalan kod. Razlika je u razini kontrole i u psihološkom doživljaju rada. Kada programer prenese zadatak na IDE okruženje s integriranim Copilotom, doživljaj se radikalno mijenja u odnosu na agentski pristup. Operater ovdje drži uzde sustava. Cursor miša diktira mjesto akcije. Čovjek inicira definicije funkcija, a umjetna inteligencija nadopunjava sintaksu liniju po liniju, vizualno prezentirajući logiku prije svake potvrde pritiskom tipke Tab ili čitanjem razlikovnog prikaza. Osjećaj kontrole proizlazi iz izravne prisutnosti svake izvršene naredbe na ekranu ispred vas. Kognitivni napor je značajan jer se stalno vrši vizualna revizija, a brzina dolaska do makroekonomskih saznanja niža je nego u agentskom pristupu uslijed izostanka pune autonomije.

Ukoliko isto pitanje postavite u agencijskom okruženju unutar terminala, poput Claude Code platforme, alat će se momentalno prebaciti u petlju autonomnog izvršavanja. Agent će na vlastitu inicijativu generirati izoliranu datoteku, samostalno instalirati potrebne zavisnosti i izvršiti analizu u pozadini bez da od vas zatraži uvid u kod prije pokretanja. Osjećat ćete se impresivno brzo, a agent će demonstrirati ogromnu sirovu moć. Međutim, ta se brzina plaća potpunim gubitkom vizualne komponente radnog toka. Agent obavlja stvari iza zatvorenih vrata crne kutije, zbog čega nedostatak interaktivnih sučelja umanjuje vašu razinu sigurnosti vezanu za procese koji se odvijaju unutar same mašine. Agentsko kodiranje je superiorno za složene cjevovode koji trebete obaviti od nule. IDE integracija briljira u svakodnevnom radu kada trebete razumjeti, modificirati i nadograđivati postojeće skripte.

2.7.0.1 Praktična vježba (15 min)

Hands-on kodiranje u Positronu s makroekonomskim podatkovnim skupom na tri razine složenosti.

Razina 1. Osnovno učitavanje podataka i vizualizacija. Napišite komentar koji jasno opisuje što trebete, primjerice `# Create a histogram of monthly inflation rates`, pustite Copilot da generira osnovni `ggplot2` kod, zatim ručno prilagodite oznake osa, boje i temu prema vašim preferencijama. Cilj je osjetiti kako linijsko predlaganje šegrtuje dok vi zadržavate kontrolu nad estetikom.

Razina 2. Filtriranje podataka i regresija s robusnim greškama. Filtrirajte podatke za Hrvatsku, procijenite OLS regresiju s jednom nezavisnom varijablom i primijenite Newey-West korekciju za

autokorelaciju. Koristite razgovorni panel da pitate Copilot što znači p-vrijednost u kontekstu vaše hipoteze i što implicira negativan predznak koeficijenta. Ovdje se Copilot transformira iz stroja za autocomplete u interaktivnog metodološkog konzultanta.

Razina 3. Panel analiza s fiksnim efektima. Proširite analizu na skupinu zemalja srednje i istočne Europe. Postavite model s fiksnim efektima koristeći paket `plm`, provedite Hausmanov test za odluku između fiksnih i slučajnih efekata, i primijenite Driscoll-Kraay robusne greške. Kombinirajte linijske sugestije za sintaksu i razgovorni panel za konceptualna pitanja o ekonometrijskoj metodologiji.

3 Blok 2.3: LLM bilježnice za eksplorativnu analizu 45 min

3.1 Konverzacijska programiranje, anatomija i arhitektura LLM bilježnica

Zamislite da sjedite u laboratoriju prirodnih znanosti i imate bilježnicu u kojoj istovremeno zapisujete pitanja koja vam padnu na pamet, lijepite fotografije grafova koje ste upravo generirali, bilježite odgovore na ta pitanja, ali ta bilježnica je istovremeno i vaš istraživački dnevnik i vaš analitički laboratorij. Ne trebate ustajati od stola da biste otišli do drugog računala i pokrenuli analizu. Ne trebate kopirati tekst iz jednog prozora u drugi. Bilježnica je istovremeno i vaš istraživački dnevnik i vaš analitički laboratorij. Upravo tako funkcioniraju LLM bilježnice (LLM Notebooks), platforme koje spajaju konverzacijsko sučelje generativne umjetne inteligencije i zatvoreno okruženje za izvršavanje koda (sandbox) u jedinstven fluidni dokument. Operater, u ulozi znanstvenika ili analitičara, zapisuje pitanja na prirodnom jeziku, dok platforma aktivno interpretira te upute, samostalno generira potreban programski kod, izvršava ga u pozadini i trenutno prikazuje rezultate.

Tri platforme pokrivaju ovaj prostor, svaka sa specifičnom tehnološkom nišom, ali im je zajednička jedna presudna karakteristika koja ih razlikuje od agencijskog kodiranja. Dizajnirane su isključivo za eksploraciju i istraživanje, ne za finalnu produkciju. Prva je ChatGPT Code Interpreter, službeno poznat kao Advanced Data Analysis, koji omogućuje korisnicima učitavanje strukturiranih i nestrukturiranih podataka direktno kroz mrežni preglednik, bez potrebe za lokalnom instalacijom Python okruženja ili upravljanjem paketima. Sustav interno pristupa standardnim analitičkim bibliotekama kao što su Pandas, NumPy, Matplotlib i SciPy. Njegova najveća prednost leži u sposobnosti slijepog učitavanja podataka gdje jezični model samostalno inferira tipove varijabli, detektira anomalije i primjenjuje metode čišćenja u zaštićenom okruženju. Druga je Claude Artifacts koji, za razliku od isključivo analitičkog pristupa usmjerenog na podatke, integrira interaktivne vizualizacije i generiranje mini-aplikacija izravno unutar razgovornog sučelja. Kod se generira i vizualno izvršava u izoliranom bočnom prozoru, omogućujući operaterima trenutnu manipulaciju parametrima generiranih korisničkih sučelja, React komponenti ili SVG grafova, potičući izrazito iterativan razvojni proces. Treća je Google Colab AI s integriranim Data Science Agentom koji koristi Gemini 2.5 Flash model direktno u klasičnom Jupyter okruženju. Nova generacija agenta u Colabu sposobna je autonomno kreirati sveobuhvatan plan analize na temelju korisnikovih namjera, izvršavati kod za čišćenje podataka, logički rasuđivati o dobivenim rezultatima te generirati statističke izvještaje. Colab AI djeluje svjesno na razini cijele bilježnice, a ne samo unutar izoliranih ćelija, zadržavajući duboki kontekst cjelokupnog radnog toka.

3.2 Kognitivni pomak u eksploratornoj analizi podataka

Ključna diferencijacija LLM bilježnica od tradicionalnog ili agencijskog kodiranja očituje se u gotovo potpunoj eliminaciji tehničkog trenja (friction). Umjesto prijavljivanja na operativni sustav, kreiranja virtualnih okruženja i rješavanja nekompatibilnosti s ovisnostima paketa, operateri se mogu fokusirati isključivo na provjeru hipoteza, vizualni pregled distribucija podataka i generiranje ranih nalaza. Ova metodologija naglašava da LLM bilježnice služe primarno kao inkubatori ideja. Zamislite ih kao istraživačke skice koje crtate olovkom prije nego što uzmete platno i boje za konačnu sliku. Nalazi dobiveni u njima moraju se translahirati i formalizirati unutar produkcijski stabilnih skripti ili agencijskih pipelineova kako bi se osigurala dugoročna skalabilnost i otpornost koda na vanjske promjene. Ne morate znati kako funkcionira svaki detalj interne arhitekture tih platformi, ali morate znati da one nisu zamjena za produkcijski kod. To su laboratoriji za ideaciju, ne tvornice za isporuku.

3.2.1 Demo: Eksploratorna analiza komunikacije središnje banke i inflacijskih očekivanja

Pet koraka koji vode od sirovog učitavanja podataka do konačnog izvoza rezultata, sve kroz razgovor na prirodnom jeziku. Kako bi prikaz bio znanstveno utemeljen, koraci su integrirani sa stvarnim makroekonomskim scenarijem temeljenim na suvremenoj financijskoj literaturi, analizom odnosa između medijske pokrivenosti komunikacije središnje banke i inflacijskih očekivanja opće populacije. Ovaj specifičan istraživački problem, često analiziran u institucijama poput Banke za međunarodna poravnanja (BIS), zahtijeva složenu analizu sentimenta nad tisućama tekstova i njihovo spajanje s panel podacima o inflacijskim očekivanjima potrošača.

Korak 1. Učitavanje i autonomni pregled podataka. Istraživač učitava CSV datoteke u platformu jednostavnim mehanizmom povlačenja i ispuštanja. Prva datoteka sadrži vremenske serije FOMC priopćenja od 1994. do 2023. godine kodiranih prema jastrebovskom (hawkish) ili golubljem (dovish) sentimentu, dok druga sadrži Indeks zajedničkih inflacijskih očekivanja (CIE). Sustav bez eksplicitne instrukcije detektira tipove varijabli, prepoznaje datumske formate te ispisuje prvih nekoliko redaka s opisom strukture. Platforma samostalno inferira da su datumi u jednom skupu mjesečni a u drugom kvartalni, te predlaže metodu agregacije.

Korak 2. Vizualni pregled distribucija. Jednim konverzacijskim promptom istraživač zahtijeva višestruke grafičke prikaze: «Generiraj histogram inflacijskih očekivanja, kutičasti dijagram medijskog sentimenta po godinama i korelacijsku matricu svih numeričkih varijabli». Sustav u nekoliko sekundi generira sve tri vizualizacije istovremeno s automatski prilagođenim oznakama osi i legendama. Analitičar trenutno uočava anomalije u distribucijama tijekom perioda kada su kamatne stope udarile o nultu donju granicu (Zero Lower Bound) te nagli skok varijance nakon uvođenja redovitih FOMC konferencija za novinare u travnju 2011. godine.

Korak 3. AI identifikacija istraživačkih pitanja. Model na temelju izračunate strukture proaktivno i autonomno predlaže tri do pet potencijalnih hipoteza za testiranje, objašnjavajući ekonomsku logiku iza svake. Na primjer, AI može uočiti fenomene opisane teorijom racionalne nepažnje (Rational Inattention), sugerirajući hipotezu «Reagiraju li potrošačka inflacijska očekivanja asimetrično i snažnije na ton

središnje banke isključivo tijekom perioda visoke medijske pokrivenosti, napuštajući stanje nepažnje?». Ovaj korak tjera istraživača da dublje razmisli o latentnim odnosima koje možda nije inicijalno percipirao.

Korak 4. Ciljana panel regresija. Istraživač opisuje glavno istraživačko pitanje u jednoj rečenici: «Predviđa li intenzitet medijske pokrivenosti komunikacije središnje banke dugoročna inflacijska očekivanja kućanstava?». AI autonomno odabira odgovarajuću ekonometrijsku metodu, generira Python kod koristeći biblioteke `statsmodels` ili `linearmodels`, izvršava ga u pozadini i prezentira rezultate. Prikazani ispis uključuje koeficijente, standardne greške i R^2 vrijednosti uz tekstualnu interpretaciju.

Korak 5. Izvoz i formalizacija rezultata. Sustav stvara konsolidiranu tablicu regresijskih rezultata prikladnu za akademske članke, formatira sve vizualizacije kao PNG slike visoke rezolucije, te nudi CSV datoteku pročišćenih, interpoliranih podataka. Ovi su formati prikladni za preuzimanje i daljnji rad unutar produkcijske skripte u agencijskom okruženju.

3.3 Strateška matrica triju paradigmi AI kodiranja

Sada prelazimo na sintezu. Tri paradigme AI kodiranja koje smo prošli kroz ovaj dan, agentsko kodiranje iz prvog bloka, IDE integrirano kodiranje iz drugog bloka i konverzacijsko kodiranje kroz bilježnice u ovom bloku, nisu međusobno zamjenjive na jednostavan način. Svaka pokriva drugačiju potrebu, operira na drugačijoj razini kontrole i zahtijeva drugačiji kognitivni angažman od vas.

Strateška matrica AI kodiranja

Dimenzija	Agencijsko (Claude Code)	IDE integrirano (Copilot, Cursor)	Konverzacijsko (LLM bilježnice)
Razina kontrole	Niska (AI samostalno vodi proces)	Visoka (čovjek piše, AI predlaže)	Srednja (čovjek opisuje, AI izvršava)
Složenost projekta	Višedatotečni repozitoriji, refaktoriranje sustava	Pojedinačne skripte, lokalno iterativno debugiranje	Jednokratna eksploratorna analiza, vizualizacije
Krivulja učenja	Srednja (CLI i sistemski prompting)	Srednja (čitanje i recenzija koda)	Niska (interakcija putem prirodnog jezika)
Reproducibilnost	Visoka (Git integracija, automatizirani commitovi)	Visoka (skripte u verzioniranom projektu)	Niska (sesije se gube, prisutna nevidljiva stanja)

Dimenzija	Agencijsko (Claude Code)	IDE integrirano (Copilot, Cursor)	Konverzacijsko (LLM bilježnice)
Idealan scenarij	Cjelokupni produkcijski pipeline od nule	Svakodnevno pisanje, testiranje, refaktoriranje	Brza provjera hipoteza, vizualni pregled distribucija
Arhitektonski fokus	Terminal (CLI) i interakcija s OS-om	Ergonomija unutar IDE-a	Interaktivnost i trenutna vidljivost u pregledniku
Primjer zadatka	Kompletna analiza od repozitorija do rezultata	Pisanje funkcije, isprobavanje, poboljšavanje	Mogu li novi podaci dati drukčiji rezultat

Ova klasifikacija služi kao strateški orijentir. Za višefazne projekte s garancijama konzistentnosti, agencijsko programiranje je optimalan izbor. Za svakodnevno testiranje malih logičkih jedinica, preferira se IDE integracija. Kada je imperativ brzina dolaženja do inicijalnih nalaza, LLM bilježnice pokazuju superiorne performanse.

3.4 Kriza reproducibilnosti, cijena interaktivnosti i automatizacije

Masovni prijelaz prema generativnoj umjetnoj inteligenciji ubrzao je inovacije, ali istovremeno eksponirao i značajno produbio krizu reproducibilnosti. Reproducibilnost, intrinzična mogućnost da nezavisni znanstvenik ponovno izvede računalni proces pod istim uvjetima i dobije potpuno identične rezultate, temelj je akademskog i inženjerskog povjerenja. Zamislite da ste kuhar koji je napravio izvrsno jelo, ali niste zapisali recept. Kada vas sljedeći tjedan netko zamoli da ponovite to jelo, sjećate se otprilike što ste stavili, ali ne znate točne količine, ne znate koliko je dugo trebalo stajati na vatri, i rezultat je svaki put malo drugačiji. Upravo to se zbiva s LLM bilježnicama i agentskim sustavima kada se ne poduzmu mjere za osiguranje ponovljivosti.

Kompjuterske bilježnice same po sebi oduvijek pate od problema skrivenog stanja (hidden state) i nelinearnog redoslijeda izvođenja. Operateri u žaru istraživanja nerijetko izvršavaju ćelije koda izvan predviđenog reda, nehotice prebrisujući ključne varijable u radnoj memoriji, pri čemu taj povijesni slijed događaja ostaje bez vidljivog traga u konačnom dokumentu. Integracija proaktivne umjetne inteligencije eksponencijalno multiplicira ovaj problem. Empirijska istraživanja na golemim uzorcima GitHub repozitorija pokazuju da više od 90 posto takvih bilježnica ne deklarira egzaktnu verziju softverskih paketa od kojih ovise. Ako se analizirana bilježnica pokuša reproducirati samo mjesec dana kasnije na drugom računalu, vanjske biblioteke mogle su biti ažurirane, imena funkcija promijenjena, a API strukture izmijenjene. Ovo neizbježno dovodi do rušenja koda, fenomena poznatog kao pomak reproducibilnosti (reproducibility drift). Nadalje, zbog inherentne stohastičnosti u generiranju teksta samih jezičnih modela, slanje istog upita u LLM bilježnicu drugi put gotovo će sigurno rezultirati tehnički drugačijom implementacijom koda.

Slični duboki problemi manifestiraju se kod implementacije terminalnih agenata. Autonomni agent sposoban je samostalno kreirati tisuće linija koda i konfigurirati cijeli projekt, ali uslijed agencijskih slijepih točaka često propušta dokumentirati preliminarne korake postavljanja okruženja. Agenti redovito zaboravljaju fiksirati

točne verzije paketa u datotekama poput `requirements.txt` jer pretpostavljaju da je lokalno okruženje već optimalno strukturirano. Rezultat je zavodljiva zamka. Projekti savršeno funkcioniraju na originalnom stroju, ali se ne mogu inicijalizirati na računalu drugog suradnika.

Tri praktične strategije štite reproducibilnost. Prvo, prisilno forsiranje deklaracije okruženja kroz sistemske upute kojima se AI alatima eksplicitno uvjetuje rigorozno fiksiranje verzija softvera (version-pinning) i obvezno korištenje sustava za kontejnerizaciju poput Dockera za svako ponuđeno rješenje. Drugo, potpuna transparentnost i provenijencija, što znači da sve interakcije sa strojem, uključujući sve povijesne promptove, cjelokupan dnevnik API poziva, svaku grešku na terminalu i modifikacije koda, moraju se trajno logirati. Treće, implementacija reproducibilnosti kao usluge (Reproducibility as a Service, RaaS) unutar cjevovoda za kontinuiranu integraciju, gdje sustavi za automatsku evaluaciju izoliraju, rekreiraju i testiraju reproducibilnost cjelokupnog AI generiranog koda u potpuno novim instancama oblaka prije nego kod dobije dozvolu za uključivanje u glavnu razvojnu granu repozitorija.

i Rizici reproducibilnosti po paradigmama		
Paradigma	Dominantni rizik	Strategija ublažavanja
LLM bilježnice	Skrivena stanja memorije, drift, nedokumentirane ovisnosti	Prisilno linearno izvođenje, automatski pinning okruženja, RaaS audit
Autonomni agenti (CLI)	Netransparentne sistemske izmjene, gubitak konteksta	Opsežna Git povijest commitova, strogo izolirana Docker okruženja
IDE AI alati	Odobranje vizualno ispravnog ali arhitektonski promašenog koda	Ljudski Code Review kao imperativ, statička analiza tipova

3.4.0.1 Praktična vježba (15 min)

Eksploratorna analiza u pet pitanja koja postupno produbljuju istraživanje. Počinjete s osnovnom deskriptivnom statistikom, primjerice «Koliko opažanja i varijabli imam u ovom skupu podataka?». Nastavljate s vizualizacijom odnosa dviju varijabli, primjerice «Kako vizualno izgleda odnos između inflacije i medijske pokrivenosti?». Zatim formuliirate hipotezu na temelju vizualnog uzorka, testirate je statističkim modelom, i završavate vizualnim sažetkom nalaza za prezentaciju. Usporedite iskustvo rada u bilježnici s onim što ste vidjeli u agentskom i IDE bloku. Gdje se osjećate sigurnije? Gdje ste brži? Gdje imate veću kontrolu?

4 Blok 2.4: Pisanje i uređivanje akademskog teksta s AI 45 min

4.1 Pisanje kao istraživačko jezgro, ne epilog

Zamislite da ste proveli tri mjeseca prikupljajući podatke, čisteći varijable, procjenjujući modele s Newey-West standardnim greškama i provjeravajući robusnost kroz specifikacijsku krivulju. Sve je na mjestu. Rezultati su

signifikantni, dijagnostički testovi prolaze, kodna baza je uredna. A onda sjednete pisati i shvatite da ne znate artikulirati što ste zapravo otkrili. Paragraf koji ste napisali zvuči kao laboratorijski dnevnik, ne kao akademski rad. Argument se rasipa, zaključci lebde bez uporišta, čitatelj gubi nit nakon drugog odlomka. Upravo ta situacija događa se svakom istraživaču barem jednom, i upravo zato posljednji blok ovog dana posvećujemo pisanju. Pisanje nije epilog istraživanja. Pisanje je dio samog istraživanja, faza koja vas prisiljava da razmislite što zaista znate, što pretpostavljate i što ne možete tvrditi. Studenti redovito podcjenjuju ovu fazu na početku rada, uvjereni da će analiza biti teška a pisanje lako. Praksa pokazuje suprotno. Pisanje konzumira nerazmjerno mnogo vremena i kognitivne energije, otkriva unutarnje kontradikcije u vašem razmišljanju i zahtijeva vrstu intelektualne zrelosti koju samo refleksivni proces može izgraditi.

U tom kontekstu umjetna inteligencija preuzima ulogu naprednog kognitivnog asistenta koji automatizira takozvane mikrozadatke. To su zadaci koji su previše repetitivni i vremenski zahtjevni za ljudske istraživače, a istovremeno presitni da bi se delegirali kolegama ili asistentima. Zamislite ih kao da imate urednog kolegu koji sjedi pokraj vas, čita svaki vaš paragraf i tiho bilježi gdje vam je argument slab, gdje ste koristili kolokvijalizam umjesto stručnog termina, gdje ste zaboravili kvalificirati tvrdnju i gdje se logički tok prekida. Taj kolega nikada ne piše umjesto vas, ali vam pomaže da pišete bolje. Ekonomska istraživanja Antona Korineka pokazuju da primjena ovakvih alata smanjuje vrijeme potrebno za izradu profesionalnih tekstova za otprilike 40 posto, uz istovremeno povećanje prosječne kvalitete i smanjenje nejednakosti u kvaliteti pisanja među autorima različitih razina iskustva. Ta brojka nije poziv na lijenost. Ona ukazuje gdje je AI najpametnije usmjeriti.

4.2 Četiri razine asistencije i etička demarkacija

Spektar AI asistencije u akademskom pisanju najjednostavnije se razumije kao rastući niz odgovornosti i istraživačkog rizika, poput ljestvi gdje svaki stupanj povećava i korist i opasnost. Prva razina obuhvaća gramatiku i pravopis. Ispravke na razini interpunkcije, deklinacija i tipfelera funkcionalno su jednake korištenju rječnika ili pravopisnog priručnika. Na ovoj razini AI sustav ni na koji način ne zadire u autorovu znanstvenu supstancu niti mijenja prirodu argumenta. On isključivo osigurava tehničku ispravnost jezičnog izričaja, što je posebno korisno za autore kojima jezik rada nije materinji. Korištenje AI-ja na ovoj razini uvijek je dopušteno i preporučljivo. Druga razina obuhvaća stilsko uređivanje i povratnu informaciju na razini formulacije. Agent čita vaš tekst i predlaže poboljšanja u strukturi argumenta, akademskom tonu i logičkom toku ideja. Može primijetiti da ste koncept spomenuli na jednom mjestu ali da bi ga bilo logičnije elaborirati u drugom poglavlju, ili da vam nedostaje prijelaz između dvaju paragrafa. Ovo je dopušteno uz transparentnost, jednako kao što je dopušteno pokazati nacrt rada kolegi i tražiti formativne komentare.

Treća razina donosi prelazak u sivu zonu i uključuje strojno parafraziranje i restrukturiranje cijelih paragrafa. Ovdje se autor oslanja na AI kako bi preoblikovao grube bilješke ili nespretno formulirane misli. Prihvatljivost ove razine snažno ovisi o kontekstu institucije i razini transparentnosti. Ako se alat koristi za stilsku elevaciju vlastitih izvornih ideja, većina institucija to tolerira. Međutim, ako se AI koristi za automatsko parafraziranje složene literature koju autor uopće nije samostalno pročitao i spoznajno usvojio, to predstavlja epistemološki prekršaj. Parafraziranje u znanosti nije mehanička zamjena riječi sinonimima. Ono je demonstracija kognitivnog usvajanja, procesuiranja tuđih koncepata kroz vlastitu analitičku prizmu. Četvrta razina, generiranje cijelih sekcija od nule bez autorovog tekstualnog ili idejnog doprinosa, akademski je neprihvatljiva bez eksplicitne deklaracije. Bez deklaracije, ova praksa je etički ekvivalentna plagiranju ili naručivanju rada od

treće strane. Autor koji prepušta generiranje ideja i argumenata stroju gubi autorstvo nad vlastitim radom. To je crvena linija.

Temeljno pravilo. AI je lektor i recenzent, nikada pisac iz sjene. Korištenje AI-ja za poboljšanje teksta koji ste vi napisali kvalitativno je različito od korištenja AI-ja za generiranje teksta umjesto vas. Ova radionica pokriva prve dvije razine asistencije, što je jednoznačno akademski prihvatljivo.

4.3 Institucionalni okviri i epistemološki rizici

Sveučilišta i međunarodne akademske organizacije ubrzano razvijaju regulativne okvire za primjenu AI alata u istraživanju. Fakultet organizacije i informatike Sveučilišta u Zagrebu donio je smjernice koje sustavno uređuju primjenu umjetne inteligencije u nastavi i studentskim radovima, zahtijevajući transparentno navođenje alata u metodologiji, uključivanje referenci u popise literature i potpisivanje izjava o izvornosti. Na globalnoj razini, Međunarodni odbor urednika medicinskih časopisa (ICMJE) i Odbor za etiku objavljivanja (COPE) jasno postavljaju da AI ne može biti naveden kao autor rada jer nema pravnu osobnost niti sposobnost preuzimanja odgovornosti za točnost i integritet sadržaja. Ljudski autori snose isključivu i nedjeljivu odgovornost za prepoznavanje pogrešaka, validaciju podataka i osiguravanje da sadržaj ne krši autorska prava.

Najveći epistemološki rizik pri korištenju AI modela u akademskom pisanju leži u pojavi halucinacija. Veliki jezični modeli nisu pretraživači činjenica već stohastički sustavi obučeni da predviđaju najvjerojatniji sljedeći niz riječi. Kao rezultat toga oni mogu generirati iznimno uvjerljive, akademski strukturirane ali potpuno izmišljene informacije, uključujući lažne citate i reference. Istraživanje Mugaanyija i suradnika iz 2024. pokazalo je da je halucinacija DOI brojeva prisutna u čak 89,4 posto slučajeva u humanističkim znanostima i u 61,8 posto slučajeva u prirodnim znanostima. Ova razlika objašnjava se strukturom znanja na kojoj su modeli trenirani jer prirodne znanosti posjeduju kruću terminološku strukturu koja ograničava prostor za pogrešku, dok humanističke znanosti omogućuju veću semantičku fleksibilnost koja povećava stopu konfabulacije. Svaki citat koji generira AI mora biti neovisno verificiran u primarnim bazama podataka poput Web of Science, Scopus ili PubMed. Nema iznimke.

Drugi ozbiljan rizik je kognitivna atrofija. Pretjerano oslanjanje na alate za generiranje i restrukturiranje teksta može dovesti do erozije temeljnih vještina pisanja i kritičkog razmišljanja, jednako kao što se mišić atrofira ako se ne koristi. Znanstveno pisanje nije samo medij za prijenos gotovih misli. Ono je dio kognitivnog procesa kroz koji se ideje oblikuju, sintetiziraju i testiraju. Empirijske ankete među studentima otkrivaju zanimljiv fenomen. Mnogi ne percipiraju korištenje AI alata kao tradicionalno varanje već kao novu kategoriju tehnološke podrške, slično korištenju kalkulatora za matematiku. Istraživanja pokazuju da institucionalne kazne imaju ograničen preventivni učinak. Ključni prediktor poštenog ponašanja su osobna etička uvjerenja studenata o tome predstavlja li AI pomoć varanje. Institucije se stoga ne smiju oslanjati isključivo na restriktivne politike i alate za detekciju AI teksta već moraju implementirati edukativne strategije koje promiču dubinsku AI pismenost. Edukacija je jača od kazne.

4.4 Šest obrazaca prompta za akademsko pisanje

Kako bi se AI zadržao u strogoj domeni lektora i recenzenta, istraživači moraju ovladati tehnikama strukturiranog komuniciranja s modelima. Promišljeni obrasci upita (prompt patterns) osiguravaju da AI djeluje isključivo unutar postavljenih restrikcija, obrađujući samo materijal koji mu je autor priskrbio.

4.4.1 Šest obrazaca za akademsko pisanje

Obrazac 1. Poboljšanje akademskog stila. Dajete modelu neformalni paragraf koji ste vi napisali, primjerice nešto poput «HNB je puno u medijima kada je inflacija visoka», a model ga vraća s akademskom terminologijom, jezikom opreznog zaključivanja i odgovarajućom strukturom argumenta. Rezultat bi mogao glasiti «Empirijski podaci impliciraju da je učestalost medijskog pojavljivanja Hrvatske narodne banke pozitivno korelirana s razinama inflacije». Vi zadržavate suštinu mišljenja, AI poboljšava formu.

Obrazac 2. Krićka analiza argumenta. Dajete paragraf iz vlastitog rada, a model identificira logićke slabosti, nedostajuće kvalifikacije, pretpostavke koje ste učinili ali niste eksplicitno naveli i potencijalne protuargumente. Model preuzima ulogu strogog sokratovskog mentora koji vas prisiljava da razmislite dublje. Ako tvrdite da medijsko izvještavanje uzrokuje inflaciju, model će vas pitati jeste li razmotrili obrnuti kauzalitet.

Obrazac 3. Generiranje prijelaza između ideja. Dajete dva paragrafa koja ne teku glatko, a model predlaže veznu rećenicu ili reformulaciju koja stvara koherentan prijelaz i smanjuje kognitivno opterećenje čitatelja.

Obrazac 4. Parafraziranje s APA atribucijom. Dajete direktni citat iz izvorne literature s toćnim podacima o autoru i godini, a model ga parafrazira zadržavajući originalno znaćenje i primjenjujući rigorozna pravila formatiranja prema APA sedmom izdanju.

Obrazac 5. Provjera konzistencije uvoda i zaključka. Dajete uvod i zaključak rada, a model analizira semantićko podudaranje. Ako ste u uvodu obećali analizu utjecaja fiskalne politike na zaposlenost ali se zaključak fokusira isključivo na monetarnu politiku, model će vas upozoriti na diskrepanciju.

Obrazac 6. Generiranje prvog nacrt a sekcije. Dajete strukturiran nacrt sa svim ključnim toćkama, a model generira bazni predloćak teksta s rezerviranim mjestima koja vi morate popuniti podacima iz vlastite analize. Ovo je Źuta zona i zahtijeva transparentnost te dubinsku autorovu reviziju.

Dva napredna obrasca zaslućuju posebnu pozornost. Obrazac obrnute interakcije (Flipped Interaction Pattern) instruiira model da preuzme ulogu ispitivaća. Vi kaćete modelu da vam postavlja pitanja jedno po jedno dok ne prikupi dovoljno informacija za strukturiranje vaće metodoloćke sekcije. Ovaj pristup sprjećava model da izmišlja sadržaj i istovremeno vas prisiljava na dublje promišljanje o vlastitom istraćivaćkom dizajnu. Obrazac predloćka (Template Pattern) nalaće modelu da izlazne podatke formatira prema zadanim strukturnim smjernicama uz zadržavanje rezerviranih mjesta koja autor ispunjava vlastitim podacima, ćime se stroju potpuno oduzima mogućnost da slobodno izmišlja sadržaj.

4.5 Korinekov okvir kognitivne automatizacije u ekonomskim znanostima

Zamislite podjelu posla u tvornici gdje svaki radnik radi ono u čemu je relativno najbolji, bez obzira na to tko je apsolutno brži. Upravo tu logiku Ricardo je opisao prije dva stoljeća, i upravo se ta logika primjenjuje na suradnju istraživača i jezičnog modela. Anton Korinek je sustavno klasificirao primjenu velikih jezičnih modela u šest domena istraživačkog rada. Prva je ideacija i povratne informacije, gdje čovjek zadržava apsolutnu prednost jer AI ne može formulirati originalne istraživačke hipoteze. Druga je pisanje, gdje čovjek vodi ali s manjom marginom jer AI izvrsno služi kao urednik. Treća je istraživanje pozadinske literature, gdje su sposobnosti gotovo izjednačene. Četvrta je obrada i analiza podataka, gdje AI već počinje biti brži. Peta je programiranje, gdje AI pokazuje signifikantnu prednost u brzini generiranja koda. Šesta je asistencija u matematičkim derivacijama, gdje AI počinje biti dominantan.

Ključna pouka ovog okvira jest da generativni modeli posjeduju neospornu komparativnu prednost u brznoj sintezi velikih količina teksta, procesiranju ulaznih podataka i generiranju inicijalnih nacрта. S druge strane, ljudski istraživači zadržavaju i moraju zadržati apsolutnu prednost u evaluaciji kvalitete, diskriminaciji logičkih argumenata, formuliranju originalnih hipoteza i donošenju završnih kritičkih sudova. Korinek sam priznaje da se oslanjao na jezične modele pri izradi vlastitih znanstvenih radova, koristeći ih za generiranje optimalnog naslova analizom apstrakta, za prevođenje apstraktnih ekonometrijskih rezultata u sažetke prilagođene široj publici i za stilsku reviziju. Vaš cilj kao istraživača je ostaviti čovjeku ono u čemu je čovjek nezamjenjiv i delegirati AI ono u čemu je AI superioran.

4.6 Četveroiteracijski protokol uređivanja

Korištenje AI alata za uređivanje teksta ne smije se percipirati kao jednokratni čarobni postupak u kojem se loš tekst jednim upitom pretvara u objavljivu kvalitetu. Pokušaj rješavanja svih problema odjednom gotovo redovito rezultira tekstualnim raspadom gdje model istovremeno optimizira makrostrukturu i mikrostrukturu, što dovodi do gubitka autorovog izvornog stila i uvođenja halucinacija.

4.6.1 Četveroiteracijski protokol uređivanja

Iteracija 1. Strukturirana evaluacija bez prepisivanja. U ovoj fazi AI modelu se strogo zabranjuje generiranje novog teksta. Dodjeljuje mu se isključiva uloga recenzenta koji evaluira nacrt na temelju četiriju parametara. Jasnoća argumentacije ocjenjuje koliko je precizno postavljena teza. Akademski stil ocjenjuje razinu formalnosti i korištenje odgovarajuće terminologije. Logički tok ocjenjuje jesu li ideje povezane koherentnim prijelazima. Gramatička korektnost ocjenjuje tehničku ispravnost jezika. Model daje kvantificiranu povratnu informaciju, primjerice «Jasnoća argumenta ocijenjena je sa 7 od 10 jer glavni argument nije orijentiran prema čitateljima bez ekonometrijskog predznanja. Logički tok ocijenjen je sa 6 od 10 jer je prijelaz između metodologije i rezultata nagao.»

Iteracija 2. Fokusirano poboljšavanje strukture. Na temelju evaluacije iz prve faze autor samostalno primjenjuje sugestije, oslanjajući se na AI za pomoć u rješavanju jedne po jedne dimenzije problema. Imperativ je započeti s makrostrukturom jer površinsko stilsko poliranje logički nekonzistentnog teksta

ne donosi nikakvu znanstvenu vrijednost. U ovoj fazi autor koristi AI kako bi definirao stručne pojmove koji su uvedeni bez konteksta i izgradio logički okvir između odlomaka.

Iteracija 3. Dodavanje akademske sofisticiranosti. Nakon strukturne stabilizacije, fokus se prebacuje na epistemološku opreznost. Umjesto dogmatskih konstatacija poput «Naš model nedvojbeno dokazuje da varijabla X uzrokuje Y», AI pomaže u integraciji kvalifikacija i ograda. Tekst se obogaćuje frazama koje odražavaju znanstvenu nesigurnost, primjerice «Rezultati sugeriraju postojanje statistički značajnog utjecaja uz pretpostavku kontroliranja za promatrane alternativne mehanizme, premda je veličina efekta relativno malena».

Iteracija 4. Finalna lingvistička i terminološka provjera. Posljednja faza posvećena je mikrolingvističkoj mehanici i terminološkoj čistoći. Rješavaju se preostali problemi s padežnim deklinacijama, sintaksom i konzistentnošću korištenja tehničke terminologije kroz cijeli dokument. Ovaj korak je obavezan kada se rad piše na hrvatskom jer globalni AI modeli pokazuju nestabilnosti pri radu s kompleksnim morfološkim strukturama slavenskih jezika.

4.7 Odabir jezičnog modela prema zadatku

Nisu svi jezični modeli jednako prikladni za sve aspekte akademskog pisanja. Claude 3.5 Sonnet posebno je valjan za kritičku analizu dugih dijelova teksta jer podržava kontekstualni prozor od 200 tisuća tokena, što znači da možete proslijediti cijeli rad sa svim referencama i bilješkama u jednom upitu. Na GPQA testu dizajniranom za evaluaciju logičkog zaključivanja na poslijediplomskoj razini, Claude ostvaruje rezultat od 59,4 posto, što ga čini prikladnijim za detekciju skrivenih pretpostavki i primjenu Obrasca 2. GPT-4o pokazuje superiorne performanse u zadacima koji zahtijevaju matematičku preciznost, s rezultatom od 76,6 posto na MATH testu, što ga čini prikladnijim za validaciju ekonometrijskih formula i derivacija. U zadacima pretraživanja literature, Claude i Gemini 1.5 Pro demonstriraju visoku osjetljivost i pronalaze gotovo sve relevantne studije ali uz veći broj lažno pozitivnih rezultata. GPT-4o i Llama 3.3 ostvaruju višu specifičnost s manje irelevantnih izvora.

i Usporedba jezičnih modela za akademsko pisanje				
Model	Kritička analiza	Matematika	Citati	Kontekst
Claude 3.5 Sonnet	Izvrсна (59.4 GPQA)	Dobra (71.1)	Osjetljiv	200K tokena
GPT-4o	Dobra (53.6)	Izvrсна (76.6)	Specifičan	128K tokena
Gemini 1.5 Pro	Dobra	Dobra	Vrlo osjetljiv	1M tokena
Llama 3.3 70B	Dobra	Dobra	Specifičan	128K tokena
Bez obzira na odabir modela, svaki predloženi citat mora biti neovisno verificiran u primarnim bazama podataka. AI je sklon generiranju bibliografskih zapisa koji izgledaju autentično ali ne postoje.				

4.8 Specifičnosti hrvatskog jezika i specijalizirani NLP alati

Svi prethodni obrasci i protokoli podrazumijevaju rad na engleskom jeziku koji dominira poljem obrade prirodnog jezika. Akademsko pisanje na hrvatskom donosi dodatne izazove jer je hrvatski bogato flektivan jezik s kompleksnim morfološkim sustavom koji obiluje padežnim deklinacijama, glagolskim konjugacijama i pravilima slaganja u rodu, broju i padežu. Kada se globalni jezični modeli prisile na dubinsko uređivanje tehničke terminologije na hrvatskom, oni još uvijek izrazito često griješe u padežnom slaganju, generirajući nespretne konstrukcije poput neusklađenosti atributa i imenica unutar složenih genitivnih sintagmi. Puko generiranje iz standardnog jezičnog modela nikada nije dovoljno za finalizaciju hrvatskog akademskog teksta.

Ekosustav CLASSLA-Stanza, razvijen u okviru CLARIN konzorcija za južnoslavenske jezike, nudi specijalizirano rješenje. Za razliku od originalnog Stanford Stanza pipelinea koji se oslanja gotovo isključivo na neuronske mreže, CLASSLA-Stanza uvodi hibridni pristup koji za lematizaciju koristi ogromne specijalizirane infleksijske leksikone za izravno pretraživanje, aktivirajući neuronske modele tek za riječi koje se ne nalaze u rječniku. Rezultati su impresivni. Empirijske analize pokazuju 98 posto manje pogrešaka pri segmentaciji rečenica, 69 posto smanjenje pogrešaka u lematizaciji i 65 posto poboljšanje u morfosintaktičkom označavanju u odnosu na bazni Stanza model. U praktičnoj primjeni, ovi alati mogu poslužiti za automatsku validaciju sintaktičke strukture u četvrtoj iteraciji uređivanja, osiguravajući da su sve tehničke riječi pravilno lematizirane i gramatički usklađene u svim padežima.

Za terminološku konzistentnost esencijalan je oslonac na Strunu, nacionalnu terminološku bazu hrvatskoga strukovnog nazivlja kojom upravlja Institut za hrvatski jezik i jezikoslovlje. Baza sustavno normira službeno nazivlje iz različitih znanstvenih polja uključujući ekonomiju. Autor u četvrtoj iteraciji može instruirati AI da uskladi sve ključne pojmove s normiranim terminima iz Strune, osiguravajući da se koncepti poput monetarne transmisije, korelacije ili inflacijskih očekivanja koriste ispravno, sustavno i u točnom padežnom obliku od prvog do posljednjeg paragrafa.

4.9 Studija slučaja s tekstom o monetarnoj politici

4.9.1 Od sirovog nacrtu do akademskog teksta kroz četiri iteracije

Polazni sirovi nacrt. «U 2022. godini inflacija je u svijetu jako narasla i centralne banke su morale reagirati kako god znaju. Ljudi su naprosto očekivali da će cijene rasti iz mjeseca u mjesec, pa su onda automatski više trošili i tako zapravo stvorili još veću inflaciju. Mediji puno pišu o tome kada je inflacija visoka, pa to medijsko ludilo zapravo uzrokuje paniku i rast cijena. Standardni modeli pokazuju da naša HNB tu ne može puno napraviti zbog prelaska na euro.»

Iteracija 1. Kritička analiza (Obrazac 2). Model ne ispravlja gramatiku niti piše novi tekst. Umjesto toga dekonstruira iznesene teze. Prva primjedba: implicitno tvrdite da medijsko izvještavanje uzrokuje rast cijena, ali jeste li razmotrili obrnuti kauzalitet? Metodološki je vjerojatnije da povišena inflacija generira pojačano medijsko izvještavanje, ne obrnuto. Druga primjedba: pojednostavljeno objašnjavate transmisijski mehanizam inflacijskih očekivanja. Prema standardnoj literaturi, ključno je razdvojiti

kratkoročna i dugoročna očekivanja. Treća primjedba: fraza «naša HNB tu ne može puno napraviti» izbjegava stručno spominjanje asimetrične transmisije monetarne politike unutar europodručja.

Iteracije 2 i 3. Strukturno poboljšanje i akademska sofisticiranost. Autor prepravlja arhitekturu argumenta uzimajući u obzir razliku između kratkoročnog i dugoročnog horizonta te ispravljajući logičku zabludu o medijskoj uzročnosti. U trećoj iteraciji koristi Obrazac 1 za podizanje tona na akademsku razinu uz integraciju jezika opreznog zaključivanja.

Rezultat nakon četiri iteracije. «Tijekom 2022. godine globalno gospodarstvo suočilo se s naglim porastom stope inflacije, što je zahtijevalo promptnu kalibraciju monetarne politike vodećih svjetskih središnjih banaka. Ekonometrijski podaci sugeriraju da su kratkoročna inflacijska očekivanja kućanstava postala iznimno osjetljiva na pojačano medijsko izvještavanje, premda je potrebno oprezno pristupiti interpretaciji smjera kauzalnosti. Metodološki je vjerojatnije da pojačani medijski fokus ne predstavlja isključivi uzrok inflatornih pritisaka već reaktivnu posljedicu na prethodno zabilježeni rast opće razine cijena. Štoviše, dok su kratkoročna očekivanja bilježila oštar uzlazni trend, dostupna istraživanja upućuju na to da su dugoročna petogodišnja inflacijska očekivanja ostala relativno stabilna i čvrsto usidrena. S obzirom na integraciju Republike Hrvatske u europodručje, mogućnosti autonomnog djelovanja HNB-a u suzbijanju asimetričnih cjenovnih šokova izravno su ovisne o širem transmisijskom mehanizmu i odlukama Europske središnje banke.»

Vidite transformaciju. Nije generirana od nule. Bila je strukturirana, iterativna, i svaki korak zahtijevao je vašu misaonu odluku. AI je poslužio kao katalizator koji je prisilio tekst na višu razinu, pravovremeno uveo sokratovsko propitivanje loših kauzalnih tvrdnji i sugerirao uključivanje temporalne dimenzije. Autor je zadržao vlasništvo nad logikom teze.

4.10 Transformirane rubrike evaluacije

Tradicionalne evaluacijske rubrike dizajnirane prije pojave jezičnih modela disproporcionalno su nagrađivale mehaničku ispravnost. Savršena gramatika, besprijekoran pravopis, tečna sintaksa. S obzirom na to da suvremeni AI alati gotovo bez greške generiraju tekst lišen gramatičkih pogrešaka, takve rubrike gube svoju diskriminatornu moć. Tekst danas može biti gramatički savršen a istovremeno analitički površan. Novi okviri za ocjenjivanje poput AI SCORE metodologije zahtijevaju pomicanje fokusa s mehanike jezika na aspekte gdje je ljudska ekspertiza nezamjenjiva. To uključuje rigorozno inzistiranje na dubinskoj točnosti informacija i obaveznoj integraciji provjerljivih referenci, oštro penaliziranje generičkog i repetitivnog jezika karakterističnog za neuređeni izlaz jezičnih modela, te dodjeljivanje najviših ocjena radovima koji demonstriraju autentičnu ljudsku analizu i sposobnost adresiranja suptilnih nijansi. Sve više institucija uvodi zahtjev za demonstracijom iterativnog procesa, primjerice kroz obvezno prilaganje refleksivnih osvrtu u kojima autor objašnjava kako je, u kojim iteracijama i zašto koristio AI alat.

4.10.0.1 Praktična vježba (15 min)

Odaberite paragraf koji ste vi napisali tijekom semestra. Idealno je da bude paragraf koji vam se čini razumnim ali ne sjajnim, vaš vlastiti uvod ili zaključak iz nekog rada ili dio laboratorijskog izvješća.

Primijenite Obrazac 1 (poboljšanje akademskog stila) i Obrazac 2 (kritička analiza argumenta). Ne trebate prolaziti sve četiri iteracije. Fokusirajte se na prve dvije. Što ste naučili o vlastitom pisanju? Koje se greške ponavljaju? Što vam je bilo iznenađenje u povratnoj informaciji? Koja kvalifikacija koju vam je AI predložio sada se čini logičnom? Cilj nije dovršiti tekst već doživjeti razliku između prvog nacrtu i verzije nakon dva kruga AI povratne informacije. To je vještina koju ćete trebati tijekom cijele karijere.

Završetak radionice

4.11 SPACE okvir, holistička evaluacija produktivnosti u eri AI-a

S eksponencijalnim porastom efikasnosti generiranja programske sintakse, tradicionalne kvantitativne metrike koje su inženjerski direktori desetljećima koristili, poput broja napisanih linija koda u satu ili broja spojenih promjena u repozitoriju, postale su beskorisne i kontraproduktivne. Umjetna inteligencija, napose agenti, može u nekoliko sekundi autonomno isporučiti tisuće linija sintaktički ispravnog ali na makro-razini potpuno arhitektonski promašenog koda. Korištenje takvih zastarjelih indikatora neminovno dovodi do pogrešnih strateških zaključaka o produktivnosti.

Zbog navedene degradacije mjerljivosti, tehnološke organizacije masovno usvajaju SPACE okvir (Satisfaction, Performance, Activity, Communication, Efficiency), strukturirani model koji su razvili istaknuti istraživači s GitHuba, instituta Microsoft Research i Sveučilišta u Victoriji, poput Nicole Forsgren. Korijeni ove filozofije sežu u pedesete godine dvadesetog stoljeća i sociotehničku teoriju sustava (STS) istraživača s Instituta Tavistock. Eric Trist i Ken Bamforth proučavali su utjecaj rane mehanizacije na produktivnost radnika u britanskim rudnicima ugljena i dokazali da uvođenje nove, efikasnije tehnologije bez uvažavanja društvenih komponenti, suradnje i zadovoljstva izoliranih radnika dugoročno rezultira padom morala i kolapsom cjelokupnog industrijskog sustava. Iste korelacije danas se primjenjuju na interakciju čovjeka i LLM agenata.

Pet dimenzija SPACE okvira prilagođenih za AI evaluaciju pokriva spektar od psihologije do arhitekture. Zadovoljstvo i dobrobit mjeri poboljšava li integracija AI alata osjećaj ispunjenosti kod inženjera eliminacijom repetitivnih zadataka ili im stvara kognitivnu iscrpljenost uslijed stalnog lova na greške proizašle iz halucinacija, jer su prema istraživanjima psihološki zadovoljni programeri do 13 posto produktivniji. Performanse i pouzdanost striktno mjere systemske ishode i kvalitetu, ne individualni output broja znakova na tipkovnici, fokusirajući se na stopu neuspješnosti promjena, systemsku pouzdanost i prosječno vrijeme oporavka od incidenata. Aktivnost uključuje mjerenje volumena radnih aktivnosti uz napredne koncepte poput TrueThroughput analitike koja pokušava normalizirati volumen uzimajući u obzir stvarnu kognitivnu složenost zahtjeva. Komunikacija i suradnja mjeri kako invazija AI asistencije utječe na sposobnost timske koordinacije i razmjenu znanja, jer ako agenti samostalno mijenjaju desetine datoteka bez jasnih komentara, dolazi do razornih sukoba između različitih ljudskih razvojnih timova. Učinkovitost i tijek rada mjeri koliko vremena inženjeri uspijevaju ostati u stanju maksimalne koncentracije, jer prebacivanje kognitivnog konteksta nakon samo jedne distrakcije košta programera i do 23 minute potpune koncentracije.

Temeljna pouka počiva na Teoriji ograničenja koja nalaže da je ukupna brzina isporuke bilo kojeg sustava determinirana njegovim najužim grlom. Primjena AI asistenata učinila je inženjere iznimno brzim proizvođačima sirovih redaka koda, što znači da brzina pisanja više apsolutno nije usko grlo. Usko grlo preselilo se u dvije kritične sfere, nizvodno na fazu rigorozne provjere koda (Code Review) i uzvodno na fazu inženjeringa zahtjeva. Ako menadžeri nisu u stanju definirati precizne kriterije prihvatljivosti, inženjeri ne posjeduju kvalitetan ulazni parametar pomoću kojeg bi optimalno usmjerili autonomnog AI agenta. Rezultat je zastrašujuće brza isporuka onoga što klijentu uopće ne treba.

4.12 Evaluacija povjerenja, sigurnosti i etičke kontrole

Dubinska akademska i industrijska istraživanja kolaborativne dinamike čovjeka i jezičnog modela otkrivaju opasan psihološki jaz između stvarne tehničke točnosti generiranog koda i njegove puke percipirane točnosti u očima programera. Operateri donose trenutne odluke o povjerenju pretežno na temelju razumljivosti i estetike generiranog prijedloga. Ukoliko agent generira kod koji je vizualno elegantan, formatiran sukladno industrijskim konvencijama te sadrži bogate komentare, ljudski mozak pokazat će snažnu pristranost da ga automatski odobri, čak i ako se unutar te impresivne fasade skrivaju sigurnosne ranjivosti ili logički propusti. Analize pokazuju da programeri, nakon temeljite kognitivne provjere, odlučuju zadržati i integrirati prosječno svega 52 posto originalno ponuđenih AI sugestija, modificirajući ili u potpunosti odbacujući ostatak. Ovo nije znak da AI sustavi ne funkcioniraju. Ovo je znak da kritički ljudski nadzor nije opcija nego imperativ.

Kao ključni akademski instrument za procjenu psihološkog oslanjanja ljudi na pametne algoritme koristi se Skala povjerenja u automatizaciju (Trust in Automation Scale, TIAS). Ova standardizirana psihometrijska metrika korisna je u identificiranju dviju patologija radnog procesa. Prva je patološko povjerenje, fenomen gdje preopterećeni operater naslijepo i bez ikakvog nadzora vjeruje stroju, što rezultira propustima u produkciji. Druga je patološko nepovjerenje, stanje u kojem skeptični operater troši enormnu količinu vremena provjeravajući apsolutno svaku trivijalnu naredbu koju model izda, čime u potpunosti anulira početne prednosti u brzini koje je AI trebao donijeti. Vaš cilj kao istraživača je pronaći sredinu, kalibrirano povjerenje koje ne zanemaruje mogućnost greške ali ne paralizira radni tok. Na tehničkoj razini sigurnosti, duboka integracija višedatotečnih AI alata nalaže stroge arhitektonske provjere, jer autonomni agenti mogu neovisno pretraživati baze podataka, pokretati skripte i izvršavati mrežne zahtjeve s privilegijama samog korisnika, što otvara pitanja privatnosti podataka i ranjivosti poznate kao posredna injekcija promptova (indirect prompt injection).

4.13 Pet ključnih poruka iz dana

Prvo. Pozicioniranje alata prema namjeni, ne prema popularnosti. Konverzijske LLM bilježnice dominiraju kao digitalni inkubatori ideja i nezamjenjivi alati za robusnu eksploratornu analizu podataka. Lokalni IDE integrirani asistenti optimalni su za visokokontrolirane taktičke iteracije kodiranja i minimizaciju prekida koncentracije. Terminalske agencijske sustavi snažni protokolima poput MCP-a trebali bi se rezervirati za dubinske arhitektonske izmjene i orkestraciju rješenja uz delegiranje zadataka višestrukim podagentima.

Drugo. Borba protiv tehnološkog duga i krize reproducibilnosti je obavezna. Iluzija brzine generativne umjetne inteligencije ne smije se odobriti na račun kompromitacije znanstvene i inženjerske robusnosti. Svaki nereproducibilan generativni kod izgrađuje latentni tehnološki dug koji će se srušiti prilikom prvog osvježavanja sustava. Prisilno inzistiranje na transparentnom bilježenju svih zavisnih uvjeta (environment pinning) i izoliranoj kontejnerizaciji (Docker) fundamentalna je disciplina.

Treće. Kvaliteta ulaza određuje kvalitetu izlaza. Precizan prompt s jasno navedenim kontekstom, s ograničenjima što agent može i ne može raditi, i s primjerima što očekujete, daje dramatično bolje rezultate od generičkog pitanja.

Četvrto. Uska grla isporuke nepovratno se sele sa sinteze redaka koda na procese strateškog planiranja, logičke arhitekture i rigorozne recenzentske validacije. Primjena obuhvatnih okvira poput SPACE-a prevenira sagorijevanje koje nastaje iz pokušaja da se primoraju iskusni profesionalci da cijeli radni dan provode u ulozi lektora nad isporukama autonomnih agenata. Mjerite brzinu rješavanja problema klijenta, ne tipkarsku aktivnost.

Peto. U svojoj konačnici, istinska vrijednost suvremenih tehnologija ne počiva u iluzornoj zamisli istiskivanja iskusnih istraživača. Njihova svrha leži u dramatičnoj elevaciji ljudskog kapaciteta za apstraktno rješavanje kompleksnih problema. Umjetna inteligencija je zauzela nišu generacije sintakse i manualne egzekucije. Ljudskim sudionicima preostaje preuzeti i usavršavati vještine strateškog usmjeravanja, dubokog semantičkog rasuđivanja i etičkog postavljanja ispravnog kursa. Programeri i analitičari budućnosti značajno će prestati nalikovati na mehaničke pisce računalnih instrukcija i zauzeti ulogu orkestratora složenih kognitivnih sustava.

Resursi za nastavak učenja

Svi materijali radionice dostupni su na GitHub repozitoriju <https://github.com/lisikic/ai-research-tools>. Promptovi koje smo koristili, primjeri podataka i vježbe mogu se koristiti kao polazna točka za vlastite projekte. Za pitanja, povratne informacije ili pojašnjenja o materijalu javite se na irubil@eizg.hr ili luka.sikic@unicath.hr.